

Structure of Desktop Computer

Definition

A Desktop Computer is an electronic computing system designed for use at a fixed location such as a home, office, school, or laboratory. It consists of several hardware components that work together to accept data as input, process it according to instructions, store information, and produce output. The main components of a desktop computer include the Central Processing Unit (CPU), Memory Unit, Input Devices, Output Devices, and Storage Devices.

The structure of a desktop computer follows the basic **Von Neumann Architecture**, where the CPU communicates with memory and input/output devices through system buses.

Main Components of a Desktop Computer

1. Input Unit

The input unit is used to enter data and instructions into the computer.

Examples:

- Keyboard
- Mouse
- Scanner
- Microphone
- Webcam

Functions:

- Accepts data from the user.
 - Converts data into machine-readable form.
 - Sends data to the CPU for processing.
-

2. Central Processing Unit (CPU)

The CPU is known as the **brain of the computer** because it controls and processes all operations.

Components of CPU

a) Arithmetic Logic Unit (ALU)

Performs arithmetic and logical operations.

Examples:

- Addition
- Subtraction

- Multiplication
- Division
- AND, OR, NOT operations

b) Control Unit (CU)

Controls and coordinates all computer activities.

Functions:

- Fetches instructions from memory.
- Decodes instructions.
- Controls data flow between components.

c) Registers

Small high-speed storage locations inside the CPU.

Examples:

- Program Counter (PC)
 - Instruction Register (IR)
 - Memory Address Register (MAR)
 - Memory Data Register (MDR)
-

3. Memory Unit

The memory unit stores data, instructions, and results.

Primary Memory

RAM (Random Access Memory)

- Temporary memory.
- Volatile in nature.
- Data is lost when power is off.

ROM (Read Only Memory)

- Permanent memory.
 - Non-volatile.
 - Stores booting instructions.
-

4. Storage Unit

Used for permanent storage of data and programs.

Examples:

- Hard Disk Drive (HDD)
- Solid State Drive (SSD)
- DVD
- Pen Drive

Advantages:

- Large storage capacity.
 - Permanent data storage.
-

5. Output Unit

Displays processed information to users.

Examples:

- Monitor
- Printer
- Speaker
- Projector

Functions:

- Converts machine-readable output into human-readable form.
 - Presents results to the user.
-

Working of a Desktop Computer

1. User enters data using input devices.
 2. Data is stored in memory.
 3. CPU fetches instructions from memory.
 4. ALU performs calculations and logical operations.
 5. Processed results are stored temporarily in memory.
 6. Output devices display the final result.
 7. Storage devices save information permanently.
-

Example

When a user types a document in MS Word:

- Keyboard acts as the Input Device.
 - CPU processes keystrokes.
 - RAM temporarily stores the document.
 - Monitor displays the text.
 - SSD/HDD saves the document permanently.
-

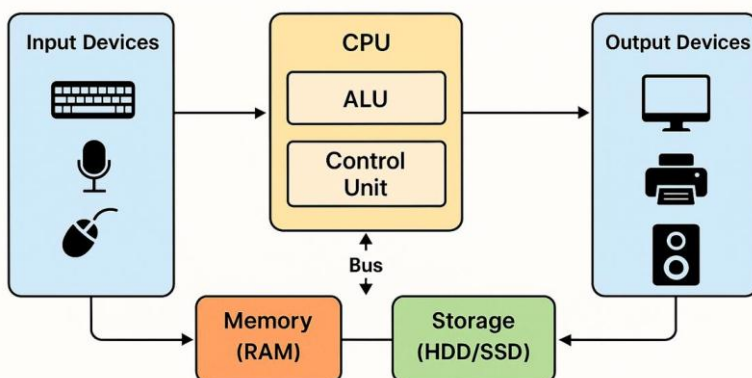
Advantages of Desktop Computer

1. High processing power.
 2. Easy hardware upgrades.
 3. Large storage capacity.
 4. Better cooling system.
 5. Cost-effective compared to laptops with similar specifications.
 6. Suitable for gaming, programming, and professional work.
-

Disadvantages of Desktop Computer

1. Not portable.
2. Requires dedicated space.
3. Needs continuous power supply.
4. More cables and peripherals.
5. Difficult to carry during travel.

Top-Level View of Computer Structure

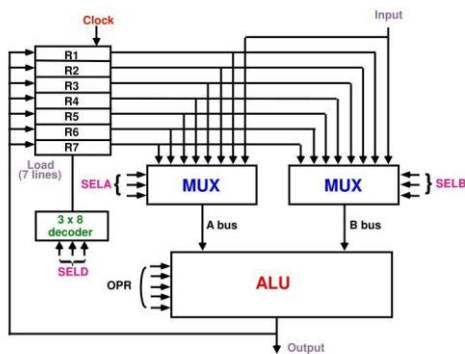


CPU: General Register Organization

Definition

General Register Organization refers to the arrangement of registers inside the CPU for storing data, instructions, addresses, and intermediate results. Registers are high-speed storage locations that help the CPU execute instructions efficiently. All registers are interconnected through a common bus system.

CENTRAL PROCESSING UNIT GENERAL REGISTER ORGANIZATION



General Register Organization

Components of General Register Organization

1. Memory Address Register (MAR)

MAR stores the address of the memory location from which data is to be read or written.

Functions:

- Holds memory addresses.
- Sends addresses to the memory unit.
- Used during memory access operations.

Example:

If data is stored at address 500, MAR will contain 500 before memory access.

Advantages

- Fast address handling.
- Improves memory access efficiency.

Disadvantages

- Stores only addresses, not data.
- Limited register size.

2. Memory Data Register (MDR)

MDR temporarily stores data transferred between CPU and memory.

Functions:

- Holds data read from memory.
- Holds data to be written into memory.

Example:

If value 25 is fetched from memory, MDR temporarily stores 25.

3. Instruction Register (IR)

IR stores the instruction currently being executed.

Functions:

- Holds fetched instruction.
- Sends instruction to Control Unit for decoding.

Example:

If instruction is ADD R1,R2, it is stored in IR before execution.

Advantages

- Faster instruction execution.
- Helps instruction decoding.

Disadvantages

- Stores only one instruction at a time.

4. Program Counter (PC)

PC contains the address of the next instruction to be executed.

Functions:

- Tracks program execution.
- Automatically increments after each instruction fetch.

Example:

If current instruction is at address 200, PC may contain 201.

Advantages

- Maintains execution sequence.
- Essential for program control.

Disadvantages

- Wrong PC value may disrupt execution.

5. Control Word

A Control Word is a binary word generated by the Control Unit to control micro-operations inside the CPU.

Functions:

- Controls data transfer.
- Activates registers and ALU operations.

Example:

A control word may command:

$R1 \leftarrow R2 + R3$

Advantages

- Efficient control of CPU operations.
- Supports microprogramming.

Disadvantages

- Complex control logic.

6. Stack Organization

A Stack is a memory structure that follows the **LIFO (Last In First Out)** principle.

Operations:

- PUSH (Insert)
- POP (Remove)

Example:

Push A

Push B

Push C

Pop $\rightarrow$ C

Advantages

- Easy function call handling.
- Supports recursion.

Disadvantages

- Limited size.
- Stack overflow may occur.

7. Instruction Format

Instruction Format specifies how instructions are represented in binary form.

Components

1. Opcode Field
2. Address Field

3. Mode Field

Example

```
+-----+-----+  
| Opcode | Address |  
+-----+-----+
```

Instruction:

ADD R1,R2

Here ADD is Opcode and R1,R2 are Operand Fields.

Advantages

- Standardized instruction representation.
- Easy decoding.

Disadvantages

- Complex formats increase hardware complexity.

8. Arithmetic Logic Unit (ALU)

ALU performs arithmetic and logical operations.

Operations

Arithmetic

- Addition
- Subtraction
- Multiplication
- Division

Logical

- AND
- OR
- NOT
- XOR

Example

$10 + 20 = 30$

Advantages

- Fast calculations.
- Core processing component.

Disadvantages

- Cannot store data permanently.

9. I/O System

I/O System enables communication between computer and external devices.

Input Devices

- Keyboard
- Mouse
- Scanner

Output Devices

- Monitor
- Printer
- Speaker

Advantages

- User interaction.
- Data exchange with external world.

Disadvantages

- Slower than CPU operations.

10. Bus Structure

A Bus is a communication pathway connecting CPU, Memory, and I/O devices.

Types of Bus

a) Data Bus

Transfers data.

b) Address Bus

Transfers addresses.

c) Control Bus

Transfers control signals.

Advantages

- Efficient communication.
- Reduced wiring complexity.

Disadvantages

- Bus congestion may occur.

11. CPU and Memory Relationship

The CPU continuously exchanges data and instructions with memory.

Working

1. PC stores instruction address.
2. MAR receives address.
3. Instruction fetched from memory.
4. IR stores instruction.
5. Control Unit decodes instruction.
6. ALU executes operation.
7. Result stored back in memory.

Advantages

- Fast processing.
- Organized execution.

Disadvantages

- Performance depends on memory speed.

Example of Instruction Execution Cycle

Instruction:

ADD R1,R2

Steps

1. PC contains address of instruction.
2. Instruction fetched into IR.
3. Control Unit decodes instruction.
4. ALU performs addition.
5. Result stored in register R1.
6. PC updated for next instruction.

Bus Structure

Definition

A **Bus Structure** is a communication pathway that connects different components of a computer system such as the CPU, Memory, and Input/Output devices. It is used to transfer data, addresses, and control signals between these components.

In simple words, a bus acts as a **highway for information transfer** inside a computer system. Without a bus, the CPU, memory, and peripheral devices would not be able to communicate with each other.

Need of Bus Structure

- To transfer data between CPU and Memory.
- To transfer addresses from CPU to Memory.
- To transfer control signals among different units.
- To reduce the number of direct connections between components.

Types of Bus Structure

1. Data Bus

Definition

The **Data Bus** is used to transfer actual data between CPU, Memory, and I/O devices.

Characteristics

- It is a **bidirectional bus** because data can travel in both directions.
- Width of the data bus determines how much data can be transferred at one time.

Example

If a CPU has a **32-bit Data Bus**, it can transfer 32 bits of data simultaneously.

Advantages

- Fast data transfer.
- Supports communication between all system components.

Disadvantages

- Limited width restricts transfer speed.

2. Address Bus

Definition

The **Address Bus** carries the address of the memory location where data is to be read from or written to.

Characteristics

- It is a **unidirectional bus**.
- Address signals move only from CPU to Memory or I/O devices.
- Determines maximum memory capacity.

Example

A 16-bit Address Bus can address:

$2^{16} = 65,536$ memory locations

Advantages

- Enables CPU to locate memory locations accurately.
- Supports large memory addressing.

Disadvantages

- Fixed width limits addressable memory space.

3. Control Bus

Definition

The **Control Bus** carries control signals and timing signals between CPU, Memory, and I/O devices.

Common Control Signals

- Read Signal
- Write Signal
- Interrupt Signal
- Clock Signal
- Reset Signal

Characteristics

- Mostly bidirectional.
- Coordinates all operations within the computer system.

Example

When CPU wants to read data from memory, it sends a **Read Signal** through the Control Bus.

Advantages

- Synchronizes system operations.
- Ensures proper execution of instructions.

Disadvantages

- Complex control circuitry required.

Working of Bus Structure

Suppose CPU wants to read data from memory location 500:

Step 1

CPU places address **500** on the Address Bus.

Step 2

CPU sends a **Read Signal** through the Control Bus.

Step 3

Memory locates address 500.

Step 4

Data stored at address 500 is transferred through the Data Bus.

Step 5

CPU receives and processes the data.

Comparison of Bus Types

Feature	Data Bus	Address Bus	Control Bus
Purpose	Transfers Data	Transfers Addresses	Transfers Control Signals
Direction	Bidirectional	Unidirectional	Bidirectional
Information	Data Values	Memory Addresses	Commands & Signals
Used By	CPU, Memory, I/O CPU to Memory/I/O CPU, Memory, I/O		

Advantages of Bus Structure

- 1. Simplifies communication between components.
- 2. Reduces wiring complexity.
- 3. Cost-effective design.
- 4. Supports efficient data transfer.
- 5. Makes system expansion easier.

Disadvantages of Bus Structure

- 1. Bus congestion may occur.
- 2. Performance decreases when many devices share the bus.
- 3. Failure of the bus can affect the entire system.
- 4. Limited bandwidth restricts speed.

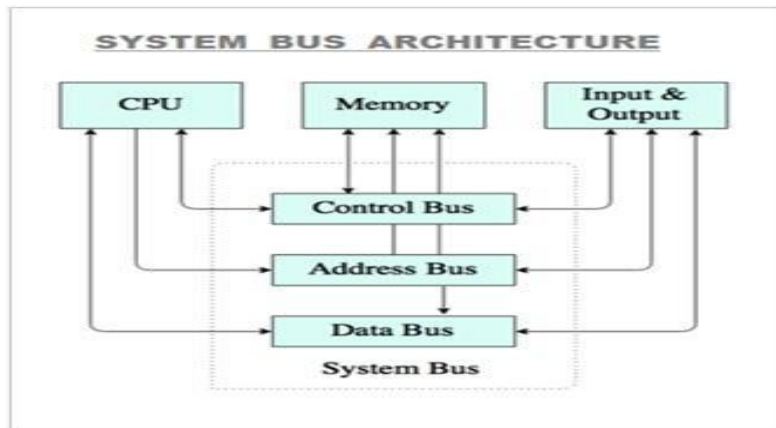
Example

When you open a file stored on the hard disk:

- **Address Bus** sends the memory location.
- **Control Bus** sends the Read command.

- **Data Bus** transfers the file data to the CPU.

Diagram



Register Transfer Language (RTL)

Definition

Register Transfer Language (RTL) is a symbolic language used to describe the transfer of data between registers, memory, and other digital system components. It represents micro-operations performed inside a computer system during instruction execution.

RTL helps in understanding how data moves within the CPU and how various operations are carried out.

Examples

- $R1 \leftarrow R2$ (Contents of R2 are transferred to R1)
- $R1 \leftarrow R2 + R3$ (Addition operation)
- $MAR \leftarrow PC$ (Program Counter content transferred to Memory Address Register)

Advantages of RTL

1. Easy representation of data transfer operations.
2. Simplifies CPU organization understanding.
3. Useful in computer architecture design.
4. Reduces complexity of micro-operation description.
5. Helps in hardware implementation.

Disadvantages of RTL

1. Not suitable for high-level programming.
2. Complex for large computer systems.
3. Requires knowledge of hardware organization.

Example

If Register R2 contains 20 and Register R1 contains 10:

$R1 \leftarrow R2$

After execution:

$R1 = 20$

$R2 = 20$

The contents of R2 are copied into R1.

Bus Transfer

Definition

Bus Transfer refers to the movement of data from one register to another through a common bus system. Instead of connecting every register directly, a bus provides a shared communication path.

A common bus reduces hardware complexity and enables efficient data transfer.

Example

Suppose:

- $R1 = 25$
- $R2 = 10$

If the operation is:

$R2 \leftarrow R1$

Then the value 25 is transferred from R1 to R2 through the common bus.

Advantages of Bus Transfer

1. Reduces wiring complexity.
2. Efficient communication between registers.
3. Cost-effective design.
4. Faster data transfer.

Disadvantages of Bus Transfer

1. Only one transfer can occur at a time on a common bus.
2. Bus congestion may occur.
3. Performance decreases with multiple users.

Memory Transfer

Definition

Memory Transfer is the process of transferring data between CPU registers and memory locations.

There are two types of memory transfer operations:

1. Memory Read

Data is transferred from memory to CPU.

Example:

$M[200] = 50$

Read Operation:

$R1 \leftarrow M[200]$

Now R1 contains 50.

2. Memory Write

Data is transferred from CPU to memory.

Example:

$R1 = 75$

Write Operation:

$M[200] \leftarrow R1$

Now memory location 200 contains 75.

Advantages of Memory Transfer

1. Enables communication between CPU and memory.
2. Supports program execution.
3. Allows storage and retrieval of data.

Disadvantages of Memory Transfer

1. Slower than register operations.
2. Memory access delays may occur.

Addressing Modes

Definition

Addressing Modes are methods used by the CPU to specify the location of operands required for instruction execution.

They determine how the effective address of data is obtained.

1. Immediate Addressing Mode

The operand value is directly specified in the instruction.

Example

MOV R1, #25

Here 25 is the actual data.

Advantages

- Fast execution.
- No additional memory access required.

Disadvantages

- Limited operand size.

2. Direct Addressing Mode

The address field contains the actual memory address of the operand.

Example

ADD R1, 500

Operand is stored at memory location 500.

Advantages

- Simple implementation.

Disadvantages

- Limited address range.

3. Indirect Addressing Mode

The address field points to another memory location that contains the effective address.

Example

If location 500 contains 800,

ADD R1, (500)

CPU first accesses location 500, then location 800.

Advantages

- Supports larger memory space.

Disadvantages

- Requires additional memory access.

4. Register Addressing Mode

Operand is stored in a CPU register.

Example

ADD R1, R2

Contents of R2 are added to R1.

Advantages

- Very fast.

- No memory access required.

Disadvantages

- Limited number of registers.

5. Register Indirect Addressing Mode

A register contains the address of the operand.

Example

R2 = 500

ADD R1, (R2)

Operand is located at memory address 500.

Advantages

- Flexible memory access.

Disadvantages

- Extra memory access needed.

6. Indexed Addressing Mode

Effective Address = Address Field + Index Register

Example

Address = 1000

Index Register = 20

Effective Address = 1020

Advantages

- Useful for arrays and tables.

Disadvantages

- Additional hardware required.

7. Relative Addressing Mode

Effective Address = Program Counter + Address Field

Example

PC = 500

Address Field = 20

Effective Address = 520

Advantages

- Supports program relocation.

Disadvantages

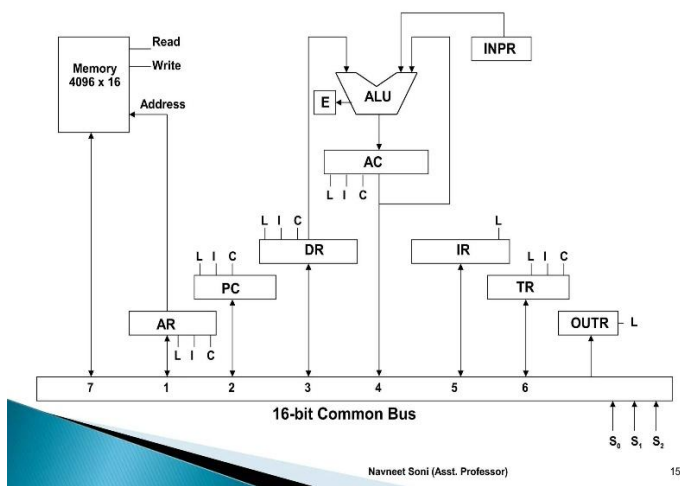
- Limited address range.

Applications of Addressing Modes

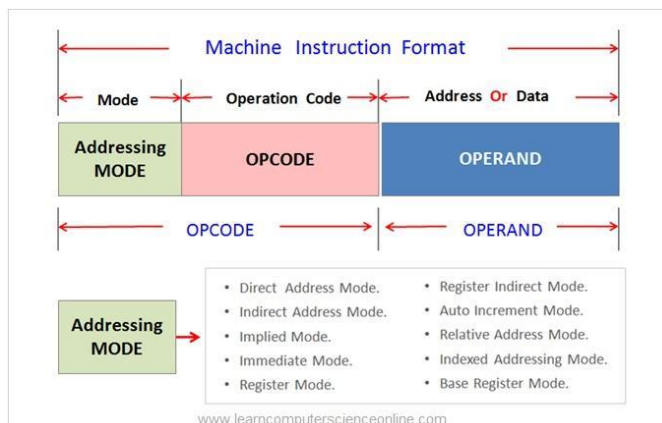
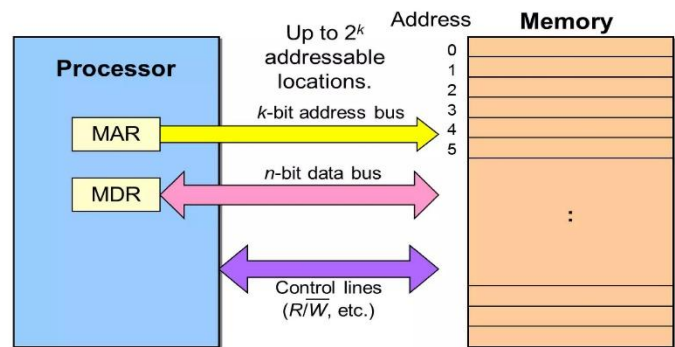
1. Array processing.
2. Loop execution.
3. Memory management.
4. Function calls.
5. Data structure implementation.

COMMON BUS SYSTEM

Registers



Data Transfer



Control Unit Organization

Definition

The **Control Unit (CU)** is an important part of the CPU that controls and coordinates all operations of the computer system. It does not perform arithmetic calculations itself but directs the flow of data between the CPU, memory, ALU, and I/O devices. The Control Unit fetches instructions from memory, decodes them, and generates control signals required for execution.

Basic Concept of Instruction

An instruction is a command given to the computer that tells the CPU what operation to perform. Every instruction consists of two main parts:

1. **Opcode (Operation Code)** – Specifies the operation to be performed.
2. **Operand** – Specifies the data or address on which the operation is to be performed.

Example

ADD R1, R2

Here:

- ADD = Opcode
- R1 and R2 = Operands

Advantages

1. Provides a systematic way to execute programs.
2. Enables communication between software and hardware.
3. Makes processing efficient and organized.

Disadvantages

1. Complex instructions require more hardware support.
2. Improper instruction design can reduce performance.

Instruction Types

1. Data Transfer Instructions

Used to transfer data from one location to another.

Example: MOV R1, R2

2. Arithmetic Instructions

Used for mathematical operations.

Example: ADD, SUB, MUL, DIV

3. Logical Instructions

Used for logical operations.

Example: AND, OR, NOT, XOR

4. Branch Instructions

Used to alter the sequence of execution.

Example: JMP, CALL, RET

5. Input/Output Instructions

Used to communicate with peripheral devices.

Example: IN, OUT

Advantages

1. Supports different types of operations.
2. Increases flexibility of CPU.
3. Enables efficient program execution.

Disadvantages

1. Large instruction sets increase complexity.
2. More hardware resources are required.

Microinstruction Format

A **Microinstruction** is a low-level instruction stored in the control memory of a microprogrammed control unit. It specifies micro-operations that must be performed during instruction execution.

Fields of a Microinstruction

1. Control Field
2. Condition Field
3. Next Address Field

Functions

- Generates control signals.
- Controls register transfers.
- Controls ALU operations.
- Determines next microinstruction.

Advantages

1. Easy modification of control logic.
2. Simplifies CPU design.
3. Supports complex instruction sets.

Disadvantages

1. Slower than hardwired control.
2. Requires additional control memory.

Fetch Cycle

The Fetch Cycle is the process of obtaining an instruction from memory.

Steps

1. Program Counter (PC) contains address of next instruction.

2. Address is transferred to Memory Address Register (MAR).
3. Instruction is fetched from memory.
4. Instruction is loaded into Instruction Register (IR).
5. Program Counter is updated.

Example

If PC = 200, the instruction stored at memory location 200 is fetched and loaded into IR.

Advantages

1. Ensures continuous instruction execution.
2. Supports automatic program sequencing.

Disadvantages

1. Memory access time affects speed.
2. Repeated for every instruction.

Execution Cycle

The Execution Cycle is the process of performing the operation specified by the fetched instruction.

Steps

1. Instruction is decoded by the Control Unit.
2. Required operands are fetched.
3. ALU performs the operation.
4. Result is stored in register or memory.
5. Status flags are updated if necessary.

Example

Instruction: ADD R1, R2

If R1 = 10 and R2 = 20

Result:

R1 = 30

Advantages

1. Performs actual computation.
2. Produces useful results.

Disadvantages

1. Complex instructions require more execution time.

2. Performance depends on CPU architecture.

Difference Between Fetch Cycle and Execution Cycle

Fetch Cycle

Retrieves instruction from memory

Uses PC, MAR, MDR, IR

Same process for every instruction

First phase of instruction cycle

Execution Cycle

Executes the instruction

Uses ALU and registers

Depends on instruction type

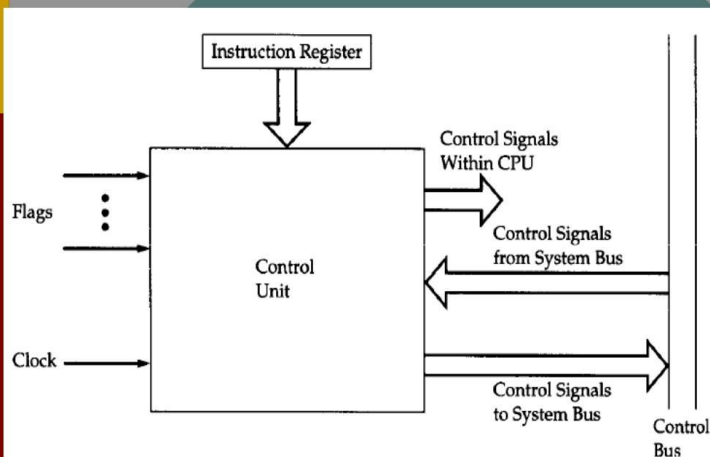
Second phase of instruction cycle

Example of Complete Instruction Cycle

Instruction: ADD R1, R2

1. PC stores instruction address.
2. Instruction fetched into IR.
3. Control Unit decodes instruction.
4. ALU performs addition.
5. Result stored in R1.
6. PC updated to next instruction.

Control Unit Block Diagram

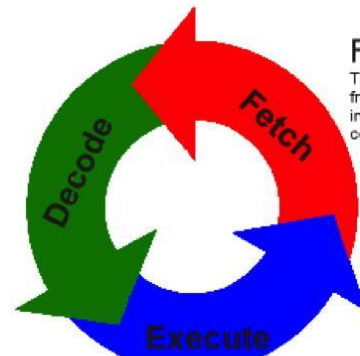


THE FETCH, DECODE & EXECUTE CYCLE.

Decode:
the bit pattern into
the instruction
register

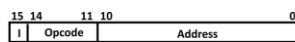
Fetch:
The next instruction
from memory and
increment the program
counter

Execute:
the action of the
instruction into the
instruction register



Micro Instruction Format

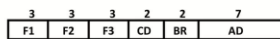
Machine instruction format



Sample machine instructions

Symbol	OP-code	Description
ADD	0000	$AC \leftarrow AC + M[EA]$
BRANCH	0001	if $(AC < 0)$ then $(PC \leftarrow EA)$
STORE	0010	$M[EA] \leftarrow AC$
EXCHANGE	0011	$AC \leftarrow M[EA], M[EA] \leftarrow AC$

Microinstruction Format



F1, F2, F3: Microoperation fields
 CD: Condition for branching
 BR: Branch field
 AD: Address field

Hardwired Control Unit

Definition

A **Hardwired Control Unit (HCU)** is a type of control unit in which the control signals are generated using fixed hardware components such as logic gates, flip-flops, decoders, counters, and control circuits. Unlike a microprogrammed control unit, it does not use control memory or microinstructions. Since the control logic is permanently built into the hardware, instructions are executed at a very high speed.

The Hardwired Control Unit is commonly used in **RISC (Reduced Instruction Set Computer)** processors because they require fast instruction execution and have simpler instruction sets.

Working of Hardwired Control Unit

The working of a Hardwired Control Unit involves the following steps:

1. The Program Counter (PC) provides the address of the next instruction.
2. The instruction is fetched from memory and stored in the Instruction Register (IR).
3. The opcode of the instruction is sent to the instruction decoder.
4. The decoder identifies the instruction type.
5. Timing and control logic generate the required control signals.
6. These control signals activate the ALU, registers, memory, and I/O devices to execute the instruction.
7. After execution, the Program Counter is updated to fetch the next instruction.

Components of Hardwired Control Unit

1. Instruction Register (IR)

Definition:

The **Instruction Register (IR)** is a special register inside the CPU that temporarily stores the instruction fetched from memory. The Control Unit reads the instruction from the IR and decodes it before execution.

Functions:

- Stores the current instruction.
- Sends the instruction to the Instruction Decoder.
- Helps in executing one instruction at a time.

Example:

If the instruction is ADD R1, R2, it is first stored in the IR.

2. Instruction Decoder

Definition:

The Instruction Decoder is a digital circuit that interprets the opcode of the instruction stored in the Instruction Register. It determines which operation is to be performed.

Functions:

- Decodes the instruction.
- Identifies the required operation.
- Generates signals for the Control Unit.

Example:

If the opcode is ADD, the decoder recognizes that an addition operation has to be performed.

3. Sequence Counter

Definition:

The Sequence Counter (SC) is a register that keeps track of the sequence of micro-operations required for instruction execution. It ensures that operations are performed in the correct order.

Functions:

- Controls the execution sequence.
- Generates timing steps.
- Coordinates different CPU operations.

Example:

During instruction execution, the Sequence Counter moves from T0 → T1 → T2 → T3 until execution is completed.

4. Timing Generator

Definition:

The Timing Generator produces timing pulses or clock signals that synchronize all operations inside the CPU. Every operation is performed according to these timing signals.

Functions:

- Generates clock pulses.

- Synchronizes CPU operations.
- Controls execution timing.

Example:

The fetch operation begins only when the Timing Generator provides the appropriate clock pulse.

5. Logic Gates

Definition:

Logic Gates are electronic circuits that generate control signals based on logical operations such as AND, OR, and NOT. They are the basic building blocks of a Hardwired Control Unit.

Functions:

- Generate control signals.
- Perform logical decision-making.
- Control data flow between components.

Example:

An AND gate may activate a register only when two required conditions are true.

6. Control Logic Circuit

Definition:

The Control Logic Circuit is a combination of logic gates, decoders, and timing circuits that generates the control signals required to execute instructions.

Functions:

- Produces control signals.
- Controls ALU, registers, and memory.
- Coordinates CPU operations.

Example:

When the instruction is LOAD, the Control Logic Circuit generates signals to read data from memory and store it in a register.

7. Control Signal Generator

Definition:

The Control Signal Generator produces the control signals that direct different components of the CPU during instruction execution.

Functions:

- Activates registers.
- Controls memory read/write operations.
- Controls ALU functions.

- **Coordinates data transfer.**

Example:

For an ADD instruction, it generates signals to enable the ALU and destination register.

Characteristics

- Control signals are generated using hardware circuits.
- No control memory is required.
- High-speed instruction execution.
- Difficult to modify after implementation.
- Suitable for processors with simple instruction sets.

Advantages

1. Very high execution speed.
2. Faster than Microprogrammed Control Unit.
3. No control memory is required.
4. Efficient for simple instruction sets.
5. Better performance in RISC processors.
6. Low instruction execution delay.

Disadvantages

1. Complex hardware design.
2. Difficult to modify or upgrade.
3. Not suitable for complex instruction sets.
4. Debugging is difficult.
5. Hardware changes are required to add new instructions.

Example

Suppose the instruction is:

ADD R1, R2

The Hardwired Control Unit immediately decodes the instruction using hardware circuits and generates control signals to:

- Read data from registers.
- Perform addition in the ALU.
- Store the result back into register R1.

Since all operations are controlled by hardware, execution is very fast.

Applications

1. RISC Processors.
2. High-speed computer systems.
3. Embedded systems.
4. Digital signal processors.
5. Real-time computing systems.

Difference Between Hardwired and Microprogrammed Control Unit

Hardwired Control Unit

Uses hardware circuits

Very high execution speed

Difficult to modify

Suitable for RISC processors

No control memory required

Hardware complexity is high

Microprogrammed Control Unit

Uses control memory and microinstructions

Slower execution speed

Easy to modify

Suitable for CISC processors

Requires control memory

Hardware complexity is lower

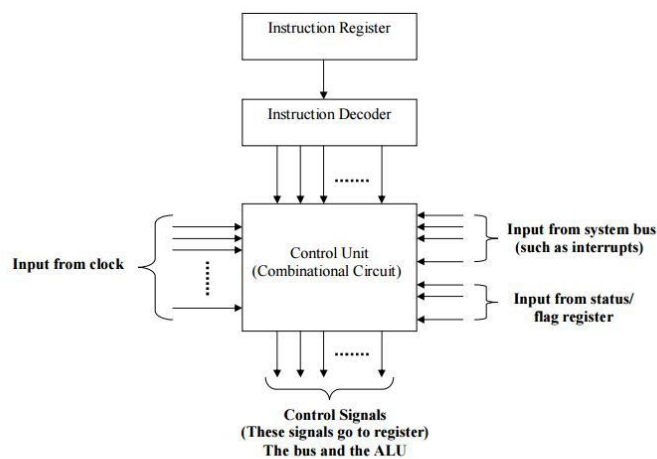


Figure (1) Hardwired Control Organization

Microprogrammed Control Unit

Definition

A **Microprogrammed Control Unit (MCU)** is a type of control unit in which control signals are generated using a set of **microinstructions** stored in a special memory called **Control Memory**. Instead of using fixed hardware circuits, it uses a microprogram to control the execution of machine instructions.

Each machine instruction is divided into a sequence of microinstructions. These microinstructions generate the necessary control signals required to perform operations inside the CPU.

The Microprogrammed Control Unit is mainly used in **CISC (Complex Instruction Set Computer)** processors because it can easily support complex instructions and can be modified without changing the hardware.

Working of Microprogrammed Control Unit

1. The instruction is fetched from memory and stored in the Instruction Register (IR).
2. The opcode of the instruction is sent to the Microprogram Sequencer.
3. The Microprogram Sequencer identifies the starting address of the required microprogram.
4. The corresponding microinstructions are fetched from the Control Memory.
5. Each microinstruction generates the required control signals.
6. The CPU performs the specified operation.
7. The next microinstruction is selected until the instruction is completely executed.

Components of Microprogrammed Control Unit

1. Microprogram Sequencer

Definition:

The **Microprogram Sequencer** is the control logic that determines the address of the next microinstruction to be executed. It controls the sequence in which microinstructions are fetched from the Control Memory.

Functions:

- Selects the next microinstruction.
- Controls the execution sequence.
- Receives opcode from the Instruction Register.
- Generates addresses for the Control Memory.

Example:

When the instruction **ADD R1, R2** is decoded, the Microprogram Sequencer selects the microprogram responsible for the ADD operation.

2. Control Memory

Definition:

Control Memory is a special memory inside the Control Unit that stores all microinstructions required to execute machine instructions. It acts as a library of microprograms used by the CPU.

Functions:

- Stores microinstructions.
- Supplies microinstructions to the Control Unit.

- Generates control signals through microinstructions.
- Supports execution of complex instructions.

Example:

If the CPU receives a **LOAD** instruction, the Control Memory provides the sequence of microinstructions needed to complete the LOAD operation.

3. Microinstruction Register (MIR)

Definition:

The **Microinstruction Register (MIR)** temporarily stores the microinstruction fetched from the Control Memory before it is executed.

Functions:

- Holds the current microinstruction.
- Sends control signals to CPU components.
- Supports execution of one microinstruction at a time.

Example:

A microinstruction fetched from the Control Memory is first stored in the MIR before controlling the ALU and registers.

4. Control Address Register (CAR)

Definition:

The **Control Address Register (CAR)** stores the address of the next microinstruction to be fetched from the Control Memory.

Functions:

- Holds the address of the current microinstruction.
- Supplies address to the Control Memory.
- Works together with the Microprogram Sequencer.

Example:

If the next microinstruction is stored at address 120, the CAR contains 120 before the Control Memory is accessed.

Advantages

1. Easy to design and implement.
2. Simple to modify or update the instruction set.
3. Suitable for complex instructions.

4. Easier debugging and maintenance.
5. Supports CISC processors.
6. Hardware complexity is lower than Hardwired Control Unit.

Disadvantages

1. Slower than Hardwired Control Unit.
2. Requires additional Control Memory.
3. Higher execution time because microinstructions must be fetched.
4. Increased memory requirement.

Applications

1. CISC processors.
2. General-purpose computers.
3. Complex CPU architectures.
4. Educational and research processors.
5. Systems requiring flexible instruction sets.

Difference Between Hardwired and Microprogrammed Control Unit

Hardwired Control Unit	Microprogrammed Control Unit
Uses hardware circuits	Uses microprograms
Faster execution	Slower execution
Difficult to modify	Easy to modify
No Control Memory	Requires Control Memory
Suitable for RISC	Suitable for CISC
High hardware complexity	Lower hardware complexity

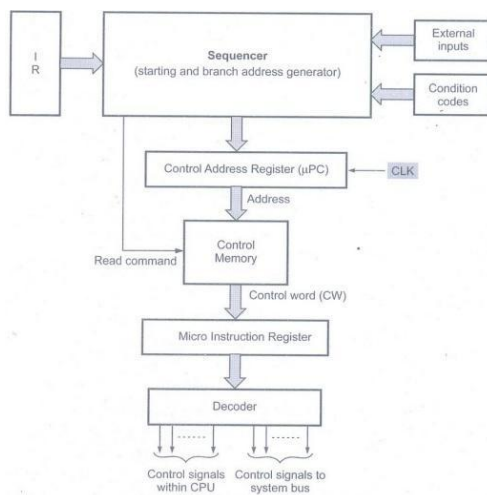


Fig. 7.6.1 Microprogrammed control unit

Sequencing and Execution of Microinstruction

Definition

Sequencing and Execution of Microinstruction is the process by which a **Microprogrammed Control Unit** fetches, selects, and executes microinstructions stored in the **Control Memory**. Each machine instruction is executed through a sequence of microinstructions, and the **Microprogram Sequencer** determines the order in which these microinstructions are executed.

In simple words, sequencing decides **which microinstruction will be executed next**, while execution performs the micro-operations specified by the current microinstruction.

Sequencing of Microinstruction

Microinstruction Sequencing is the process of selecting the address of the next microinstruction from the Control Memory. This task is performed by the **Microprogram Sequencer**.

Steps of Microinstruction Sequencing

1. The machine instruction is fetched into the **Instruction Register (IR)**.
2. The opcode is decoded.
3. The Microprogram Sequencer determines the starting address of the corresponding microprogram.
4. The address is loaded into the **Control Address Register (CAR)**.
5. The required microinstruction is fetched from the **Control Memory**.
6. After execution, the sequencer selects the address of the next microinstruction.
7. The process continues until the complete machine instruction is executed.

Execution of Microinstruction

Microinstruction Execution is the process in which the fetched microinstruction generates control signals to perform micro-operations inside the CPU.

Steps of Microinstruction Execution

1. The microinstruction is fetched from the Control Memory.
2. It is stored in the **Microinstruction Register (MIR)**.
3. The control field of the microinstruction generates the required control signals.
4. The CPU performs operations such as register transfer, ALU operation, or memory access.
5. The Microprogram Sequencer selects the next microinstruction.
6. The process repeats until all microinstructions of the machine instruction are completed.

Components Involved

1. Control Memory

Definition:

Stores all the microinstructions required to execute machine instructions.

Function:

- Stores microprograms.
- Supplies microinstructions to the CPU.

2. Control Address Register (CAR)**Definition:**

Stores the address of the next microinstruction.

Function:

- Provides the address to the Control Memory.
- Helps in fetching the next microinstruction.

3. Microinstruction Register (MIR)**Definition:**

Temporarily stores the fetched microinstruction before execution.

Function:

- Holds the current microinstruction.
- Sends control signals to CPU components.

4. Microprogram Sequencer**Definition:**

Determines the sequence of microinstructions.

Function:

- Selects the next microinstruction address.
- Controls the execution flow.

Example

Suppose the CPU executes the instruction:

ADD R1, R2

The sequence is:

- The instruction is fetched into the IR.
- The Microprogram Sequencer identifies the ADD microprogram.
- The first microinstruction is fetched from the Control Memory.
- Control signals are generated to read R1 and R2.
- The ALU performs the addition.

- The result is stored back in R1.
- The final microinstruction ends the execution and the CPU fetches the next machine instruction.

Advantages

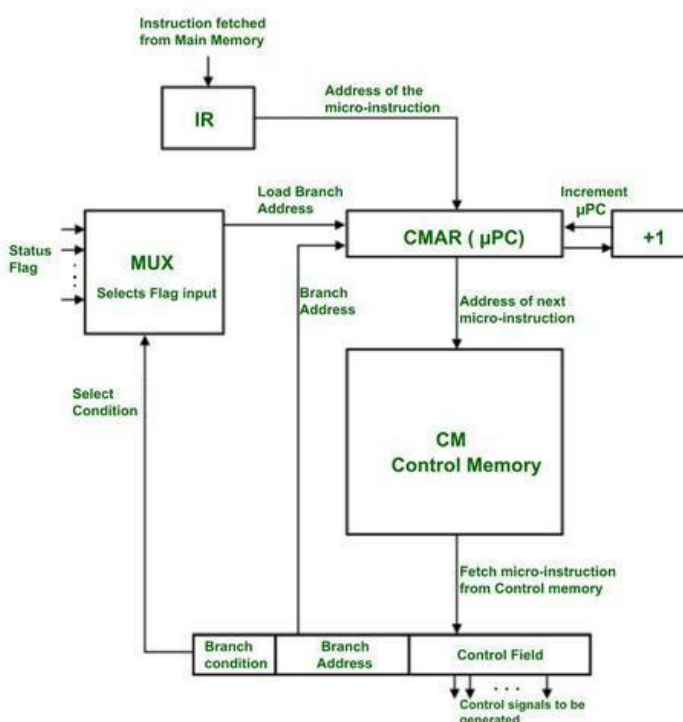
1. Easy to modify the control logic.
2. Supports complex instructions.
3. Simplifies CPU design.
4. Easy debugging and maintenance.
5. Flexible execution of instructions.

Disadvantages

1. Slower than Hardwired Control Unit.
2. Requires additional Control Memory.
3. More memory is needed to store microprograms.
4. Increases instruction execution time.

Applications

1. CISC Processors.
2. General-purpose computers.
3. Complex CPU architectures.
4. Educational processors.
5. Embedded control systems requiring flexible instruction execution.



Computer Arithmetic: Addition and Subtraction

Definition

Computer Arithmetic is the branch of Computer Organization and Architecture (COA) that deals with arithmetic operations such as **addition, subtraction, multiplication, and division** using binary numbers. Since computers understand only binary digits (0 and 1), all arithmetic operations are performed in binary form by the **Arithmetic Logic Unit (ALU)**.

Among all arithmetic operations, **addition and subtraction** are the most fundamental because multiplication and division are also performed using repeated addition and subtraction.

Binary Addition

Definition

Binary Addition is the process of adding two binary numbers according to binary arithmetic rules. The ALU performs this operation using binary adders such as Half Adder and Full Adder.

Rules of Binary Addition

- $0 + 0 = 0$
- $0 + 1 = 1$
- $1 + 0 = 1$
- $1 + 1 = 10$ (Sum = 0, Carry = 1)
- $1 + 1 + 1 = 11$ (Sum = 1, Carry = 1)

Example

Add two binary numbers:

```
  1010
+ 0110
-----
 10000
```

Explanation:

- $1010 = 10$ (Decimal)
- $0110 = 6$ (Decimal)
- Result = $10000 = 16$ (Decimal)

Advantages of Binary Addition

1. Simple and fast operation.
2. Performed directly by the ALU.
3. Requires less hardware complexity.
4. High processing speed.

5. Essential for all arithmetic calculations.

Disadvantages of Binary Addition

1. Carry propagation may slow execution.
2. Overflow may occur with limited bits.
3. Large binary numbers are difficult for humans to read.

Binary Subtraction

Definition

Binary Subtraction is the process of subtracting one binary number from another. It can be performed using **borrow method** or **2's complement method**. Modern computers mainly use the **2's complement method** because it is faster and easier to implement in hardware.

Rules of Binary Subtraction

- $0 - 0 = 0$
- $1 - 0 = 1$
- $1 - 1 = 0$
- $0 - 1 = 1$ (Borrow from the next higher bit)

Example (Borrow Method)

```
1010
- 0011
-----
0111
```

Explanation:

- $1010 = 10$ (Decimal)
- $0011 = 3$ (Decimal)
- Result = $0111 = 7$ (Decimal)

Subtraction Using 2's Complement

Instead of borrowing, computers usually perform subtraction by adding the **2's complement** of the subtracted number.

Steps:

1. Find the 1's complement.
2. Add 1 to obtain the 2's complement.
3. Add it to the first number.
4. Ignore the final carry (if generated).

Example

Subtract **5 from 9**.

- $9 = \mathbf{1001}$
- $5 = \mathbf{0101}$

1's Complement of 0101 = **1010**

2's Complement = **1011**

Now add:

```
  1001
+ 1011
-----
 10100
```

Ignoring the carry gives:

0100

Therefore, **$9 - 5 = 4$** .

Advantages of Binary Subtraction

1. Efficient implementation using 2's complement.
2. No separate subtraction hardware is required.
3. Faster execution in modern processors.
4. Reduces circuit complexity.

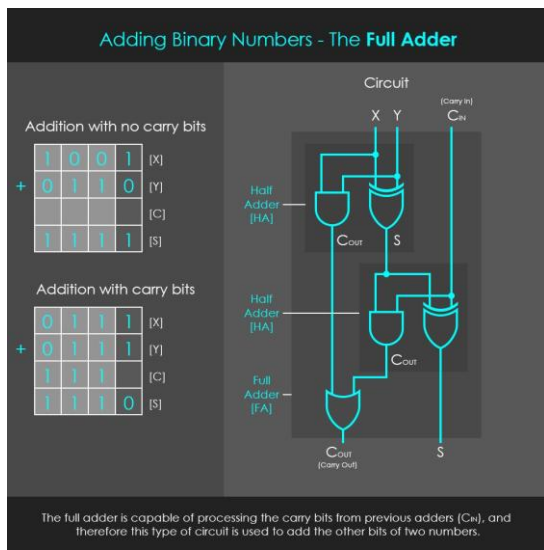
Disadvantages of Binary Subtraction

1. Borrow method is slower.
2. Overflow may occur.
3. Requires understanding of complement representation.

Applications

1. Arithmetic calculations in the ALU.
2. Scientific and engineering computations.
3. Banking and financial software.
4. Digital signal processing.
5. Computer graphics and gaming.
6. Embedded systems and microprocessors.

Diagram



Half Adder

Definition

A **Half Adder** is a combinational digital circuit that adds **two single-bit binary numbers** and produces **two outputs: Sum (S) and Carry (C)**. It is called a "Half Adder" because it cannot add a carry input from a previous stage.

Inputs and Outputs

- Inputs: A, B
- Outputs: Sum (S), Carry (C)

Boolean Expressions

- Sum (S) = $A \oplus B$ (XOR)
- Carry (C) = $A \cdot B$ (AND)

Truth Table

A B Sum (S) Carry (C)

0 0 0 0

0 1 1 0

1 0 1 0

1 1 0 1

Example

Add two binary bits:

A = 1

B = 1

Result:

- Sum = 0
- Carry = 1

Therefore, **1 + 1 = 10** (Binary).

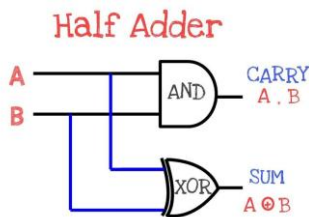
Advantages

1. Simple circuit design.
2. Requires fewer logic gates.
3. Fast operation.
4. Easy to implement.

Disadvantages

1. Cannot accept Carry Input.
2. Suitable only for single-bit addition.
3. Cannot be used directly for multi-bit addition.

Diagram



Full Adder

Definition

A **Full Adder** is a combinational digital circuit that adds **three binary inputs** (A, B, and Carry Input) and produces **two outputs: Sum (S) and Carry Output (Cout)**. It is widely used for adding multi-bit binary numbers.

Inputs and Outputs

- Inputs: A, B, Cin
- Outputs: Sum (S), Carry Out (Cout)

Boolean Expressions

- Sum = $A \oplus B \oplus \text{Cin}$
- Carry = $AB + \text{BCin} + \text{ACin}$

Truth Table

A B Cin Sum Cout

0 0 0 0 0

0 0 1 1 0

0 1 0 1 0

0 1 1 0 1

1 0 0 1 0

1 0 1 0 1

1 1 0 0 1

1 1 1 1 1

Example

A = 1

B = 1

Cin = 1

Result:

- Sum = 1
- Carry = 1

Therefore, $1 + 1 + 1 = 11$ (Binary).

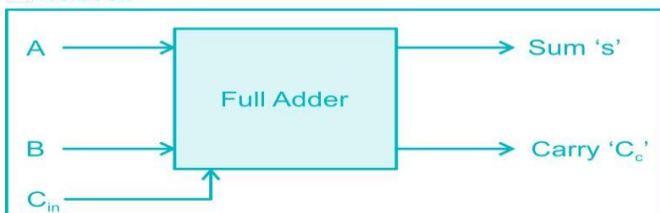
Advantages

1. Accepts Carry Input.
2. Suitable for multi-bit addition.
3. High accuracy.
4. Used in processors and ALU.

Disadvantages

1. More complex than Half Adder.
2. Requires more logic gates.
3. Higher hardware cost.

testbook



Half Subtractor

Definition

A **Half Subtractor** is a combinational digital circuit that subtracts **one single-bit binary number from another** and produces **Difference (D)** and **Borrow (B)** as outputs. It cannot accept a borrow from the previous stage.

Inputs and Outputs

- Inputs: A, B
- Outputs: Difference (D), Borrow (B)

Boolean Expressions

- **Difference = $A \oplus B$**
- **Borrow = $A'B$**

Truth Table

A B Difference Borrow

0	0	0	0
0	1	1	1
1	0	1	0
1	1	0	0

Example

A = 1

B = 0

Result:

- Difference = 1
- Borrow = 0

Advantages

1. Simple design.
2. Less hardware required.
3. Easy to implement.

Disadvantages

1. Cannot accept Borrow Input.
2. Not suitable for multi-bit subtraction.

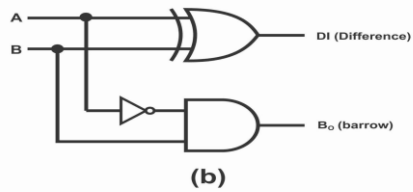
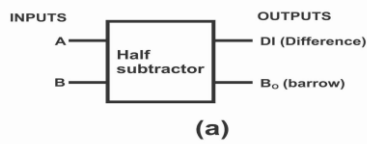


Fig 3.12 Half subtractor. (a) Block symbol. (b) Logic diagram.

Full Subtractor

Definition

A **Full Subtractor** is a combinational digital circuit that subtracts **three binary inputs** (A, B, and Borrow Input) and produces **Difference (D)** and **Borrow Output (Bout)**. It is used for multi-bit binary subtraction.

Inputs and Outputs

- Inputs: A, B, Bin
- Outputs: Difference (D), Borrow Out (Bout)

Boolean Expressions

- $\text{Difference} = A \oplus B \oplus \text{Bin}$
- $\text{Borrow} = A'B + A'\text{Bin} + B\text{Bin}$

Truth Table

A B Bin Difference Borrow Out

0	0	0	0	0
0	0	1	1	1
0	1	0	1	1
0	1	1	0	1
1	0	0	1	0
1	0	1	0	0
1	1	0	0	0
1	1	1	1	1

Example

A = 1

B = 0

Bin = 1

Result:

- Difference = 0
- Borrow Out = 0

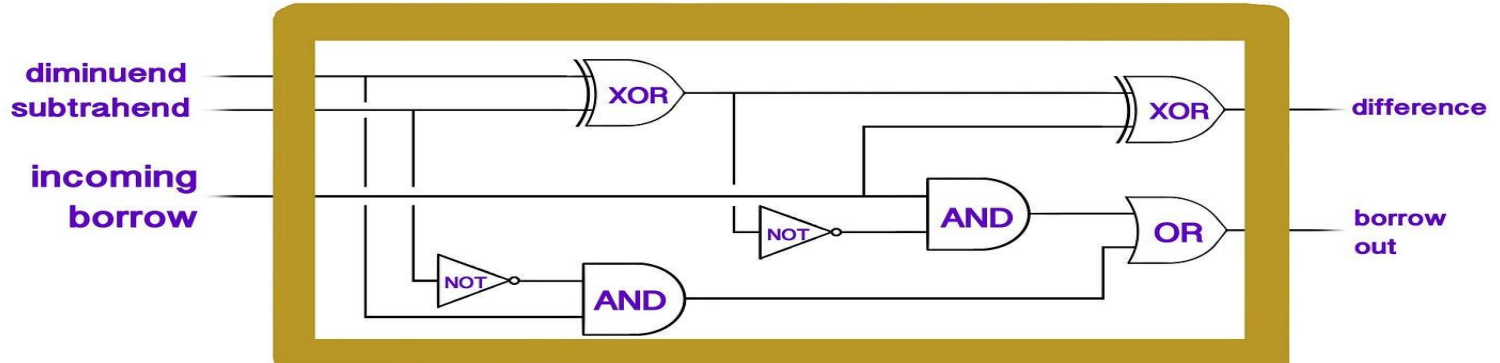
Advantages

1. Supports Borrow Input.
2. Suitable for multi-bit subtraction.
3. High accuracy.
4. Used in ALU and processors.

Disadvantages

1. More complex than Half Subtractor.
2. Requires additional logic gates.
3. Higher hardware complexity.

full subtractor



Two's Complement Representation

Definition

Two's Complement Representation is the most widely used method for representing **signed binary numbers** (both positive and negative) in computer systems. It allows the computer to perform addition and subtraction using the same hardware, making arithmetic operations simpler and faster.

In this method, **positive numbers are represented in their normal binary form**, while **negative numbers are represented by taking the Two's Complement of their corresponding positive numbers**.

Steps to Find Two's Complement

1. Write the binary representation of the positive number.

2. Find the **One's Complement** by changing all **0s to 1s** and **1s to 0s**.
3. Add **1** to the One's Complement.
4. The result is the **Two's Complement** representation.

Example

Find the Two's Complement of 5.

Step 1: Binary of 5 = **0101**

Step 2: One's Complement = **1010**

Step 3: Add 1

$1010 + 1 = \mathbf{1011}$

Therefore, the **Two's Complement of 5 is 1011**, which represents **-5** in a 4-bit system.

Representation of Positive and Negative Numbers

Decimal Number Binary (4-bit) Two's Complement Representation

+5	0101	0101
-5	0101	1011
+3	0011	0011
-3	0011	1101
+7	0111	0111
-7	0111	1001

Range of Two's Complement Numbers

For an **n-bit** binary number:

- **Minimum Value** = -2^{n-1}
- **Maximum Value** = $2^{n-1} - 1$

Example

For a **4-bit** system:

- **Minimum** = **-8**
- **Maximum** = **+7**

Therefore, the range is **-8 to +7**.

Advantages

1. Easy implementation of binary addition and subtraction.

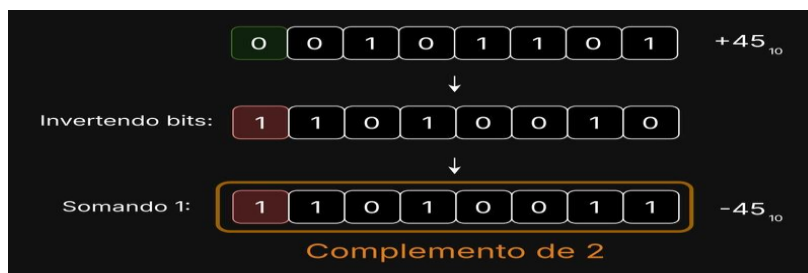
2. Only one representation for zero.
3. Eliminates the need for separate subtraction circuits.
4. Faster arithmetic operations in the ALU.
5. Widely used in modern computers and processors.
6. Simplifies hardware design.

Disadvantages

1. Difficult for beginners to understand.
2. Overflow may occur when the result exceeds the available bit range.
3. Extra care is required while converting negative numbers.

Applications

1. Representation of signed integers in computers.
2. Binary subtraction using addition.
3. Arithmetic Logic Unit (ALU) operations.
4. Microprocessors and microcontrollers.
5. Digital computers and embedded systems.



Signed Addition and Subtraction

Definition

Signed Addition and Subtraction is the process of performing arithmetic operations on **signed binary numbers** (positive and negative numbers). In computers, these operations are usually performed using the **Two's Complement Representation**. This method allows the CPU to use the same hardware for both addition and subtraction, making the arithmetic process faster and more efficient.

The **Arithmetic Logic Unit (ALU)** performs signed addition and subtraction while checking for **overflow** to ensure the result is within the allowable range.

Rules for Signed Addition

Case 1: Positive + Positive

- Add the binary numbers normally.
- The result remains positive.

- Overflow occurs only if the result exceeds the maximum range.

Example:

$$(+5) + (+3)$$

$$0101 + 0011 = \mathbf{1000} \text{ (Overflow in a 4-bit signed system)}$$

Case 2: Negative + Negative

- Add the Two's Complement representations of both numbers.
- The result is negative.
- Overflow may occur if the result is beyond the negative range.

Example:

$$(-3) + (-2)$$

$$1101 + 1110 = \mathbf{1011}$$

$$\text{Result} = \mathbf{-5}$$

Case 3: Positive + Negative

- Add the positive number and the Two's Complement of the negative number.
- The sign of the result depends on the larger magnitude.

Example:

$$(+7) + (-3)$$

$$0111 + 1101 = \mathbf{0100}$$

$$\text{Result} = \mathbf{+4}$$

Rules for Signed Subtraction

Signed subtraction is performed by **adding the Two's Complement of the subtracted number**.

Steps

1. Write the binary representation of both numbers.
2. Find the Two's Complement of the number to be subtracted.
3. Add it to the first number.
4. Ignore the carry (if generated).
5. Check for overflow.

Example 1

$$(+9) - (+5)$$

Binary:

$$9 = \mathbf{1001}$$

$$5 = \mathbf{0101}$$

Two's Complement of 5:

- One's Complement = 1010
- Add 1 = **1011**

Now add:

$$1001 + 1011 = \mathbf{10100}$$

Ignore the carry.

$$\text{Result} = \mathbf{0100} = \mathbf{+4}$$

Example 2

$$(+4) - (+7)$$

Binary:

$$4 = \mathbf{0100}$$

$$7 = \mathbf{0111}$$

Two's Complement of 7:

- One's Complement = 1000
- Add 1 = **1001**

Now add:

$$0100 + 1001 = \mathbf{1101}$$

$$\text{Result} = \mathbf{-3}$$

Overflow in Signed Arithmetic

Overflow occurs when the result of an arithmetic operation exceeds the range that can be represented by the available number of bits.

Overflow Conditions

- Adding two positive numbers gives a negative result.

- Adding two negative numbers gives a positive result.

Example

$$(+7) + (+3)$$

$$0111 + 0011 = \mathbf{1010}$$

In a 4-bit signed system, this indicates **overflow** because +10 cannot be represented.

Advantages

1. Supports both positive and negative numbers.
2. Uses the same hardware for addition and subtraction.
3. Faster arithmetic operations using Two's Complement.
4. Simplifies ALU design.
5. Eliminates the need for separate subtraction circuits.

Disadvantages

1. Overflow detection is necessary.
2. Limited range depends on the number of bits.
3. Two's Complement representation may be difficult for beginners.

Applications

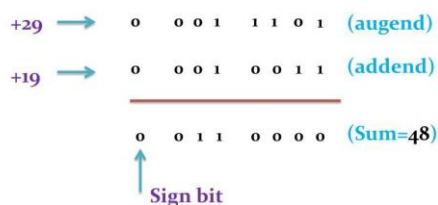
1. Arithmetic Logic Unit (ALU).
2. Microprocessors.
3. Digital computers.
4. Embedded systems.
5. Scientific and engineering calculations.
6. Banking and financial applications.

Diagram

Case 1

Two positive numbers

Consider the addition of +29 and +19



Multiplication and Division

Definition

Multiplication and Division are two important arithmetic operations performed by the **Arithmetic Logic Unit (ALU)** in a computer. Since computers work with binary numbers, these operations are carried out using **binary arithmetic**. Binary multiplication is based on repeated addition, while binary division is based on repeated subtraction. These operations are essential for scientific calculations, data processing, graphics, and many other applications.

Binary Multiplication

Definition

Binary Multiplication is the process of multiplying two binary numbers according to binary multiplication rules. It is similar to decimal multiplication, but it uses only the digits **0** and **1**.

Rules of Binary Multiplication

- $0 \times 0 = 0$
- $0 \times 1 = 0$
- $1 \times 0 = 0$
- $1 \times 1 = 1$

Example

Multiply 101×11

$$\begin{array}{r} 101 \\ \times 11 \\ \hline 101 \\ + 1010 \\ \hline 1111 \end{array}$$

Verification:

- $101_2 = 5_{10}$
- $11_2 = 3_{10}$
- $1111_2 = 15_{10}$

Therefore,

$$5 \times 3 = 15$$

Working of Binary Multiplication

1. Multiply each bit of the multiplier with the multiplicand.
2. Shift the partial product one position to the left after each step.
3. Add all partial products.

4. The final result is the binary product.

Advantages of Binary Multiplication

1. Easy to implement using digital circuits.
2. Faster processing by the ALU.
3. Suitable for all binary arithmetic operations.
4. Used in processors and digital systems.

Disadvantages of Binary Multiplication

1. Large binary numbers require more computation time.
2. Overflow may occur if the result exceeds the available bit size.
3. Hardware complexity increases for high-speed multiplication.

Binary Division

Definition

Binary Division is the process of dividing one binary number (Dividend) by another binary number (Divisor). It follows the same procedure as decimal long division but uses only binary digits.

Terms Used

- **Dividend:** Number to be divided.
- **Divisor:** Number by which division is performed.
- **Quotient:** Result of division.
- **Remainder:** Value left after division.

Example

Divide **1100** by **10**

```
      110
    -----
10 ) 1100
   -10
   ----
     10
    -10
    ----
     00
```

Verification:

- $1100_2 = 12_{10}$
- $10_2 = 2_{10}$

- Quotient = $110_2 = 6_{10}$

Therefore,

$$12 \div 2 = 6$$

Working of Binary Division

1. Compare the divisor with the leftmost bits of the dividend.
2. If possible, subtract the divisor.
3. Write **1** in the quotient; otherwise write **0**.
4. Bring down the next bit.
5. Repeat until all bits are processed.

Advantages of Binary Division

1. Accurate arithmetic operation.
2. Easy implementation in digital hardware.
3. Useful for scientific and engineering computations.
4. Supports processor arithmetic operations.

Disadvantages of Binary Division

1. Slower than binary addition.
2. Requires more hardware resources.
3. More complex than multiplication.

Applications

1. Arithmetic Logic Unit (ALU).
2. Microprocessors and Microcontrollers.
3. Scientific Computing.
4. Digital Signal Processing.
5. Computer Graphics.
6. Embedded Systems.
7. Financial and Business Applications.

Diagram



Binary Division Example

Binary Multiplication

■ **Example 2:** $00010111_2 \times 00000011_2 = 01000101_2$

$$\begin{array}{r}
 00010111 = 23_{(base\ 10)} \\
 \times 00000011 = 3_{(base\ 10)} \\
 \hline
 00010111 \\
 00010111 \\
 \hline
 00100101 = 69_{(base\ 10)}
 \end{array}$$

carries

□ Subtract the divisor at each position, if possible...

$$\begin{array}{r}
 \text{Quotient} \leftarrow 110 \\
 \text{Divisor} \rightarrow 10 \overline{) 1101} \leftarrow \text{Dividend} \\
 \underline{-10} \\
 10 \\
 \underline{-10} \\
 01 \\
 \underline{-0} \\
 1 \leftarrow \text{Remainder}
 \end{array}$$

Check:
 $13/2 = 6 \text{ r } 1 \checkmark$

3/12/2014

M. Frank, EEL3705 Digital Logic, Fall 2006

23

Booth's Algorithm

Definition

Booth's Algorithm is an efficient multiplication algorithm used to multiply **signed binary numbers (2's Complement representation)**. It reduces the number of addition and subtraction operations, making multiplication faster.

Algorithm

1. Initialize:
 - $A = 0000$
 - $M = \text{Multiplicand}$
 - $Q = \text{Multiplier}$
 - $Q_{-1} = 0$
2. Check (Q_0, Q_{-1}) :
 - 00 or $11 \rightarrow \text{No operation}$
 - $01 \rightarrow A = A + M$
 - $10 \rightarrow A = A - M$
3. Perform Arithmetic Right Shift.
4. Repeat for n bits.
5. Final result = AQ .

Solved Example

Multiply $(+5) \times (+3)$

Step 1: Binary Representation

Multiplicand (M) = 0101

Multiplier (Q) = 0011

Negative Multiplicand (-M)

2's Complement of 0101

0101

1010 (1's Complement)

1011 (+1)

Therefore,

-M = 1011

Initial Values

Register Value

A 0000

Q 0011

Q₋₁ 0

M 0101

-M 1011

Booth's Iteration Table

Cycle	Q ₀ Q ₋₁	Operation	A	Q	Q ₋₁
Initial	-	Initial Values	0000	0011	0
1	10	A = A – M	1011	0011	0
		Arithmetic Right Shift	1101	1001	1
2	11	No Operation	1101	1001	1
		Arithmetic Right Shift	1110	1100	1
3	01	A = A + M	0011	1100	1
		Arithmetic Right Shift	0001	1110	0
4	00	No Operation	0001	1110	0
		Arithmetic Right Shift	0000	1111	0

Final Result

AQ =

00001111

Decimal Value

00001111 = 15

Therefore,

$$5 \times 3 = 15$$

Another Example (Negative Number)

Multiply

$$(+5) \times (-3)$$

Binary Representation

+5 = 0101

-3 = 1101

After applying Booth's Algorithm, the final result is

11110001

This is the **2's Complement representation of -15**.

Therefore,

$$5 \times (-3) = -15$$

Advantages

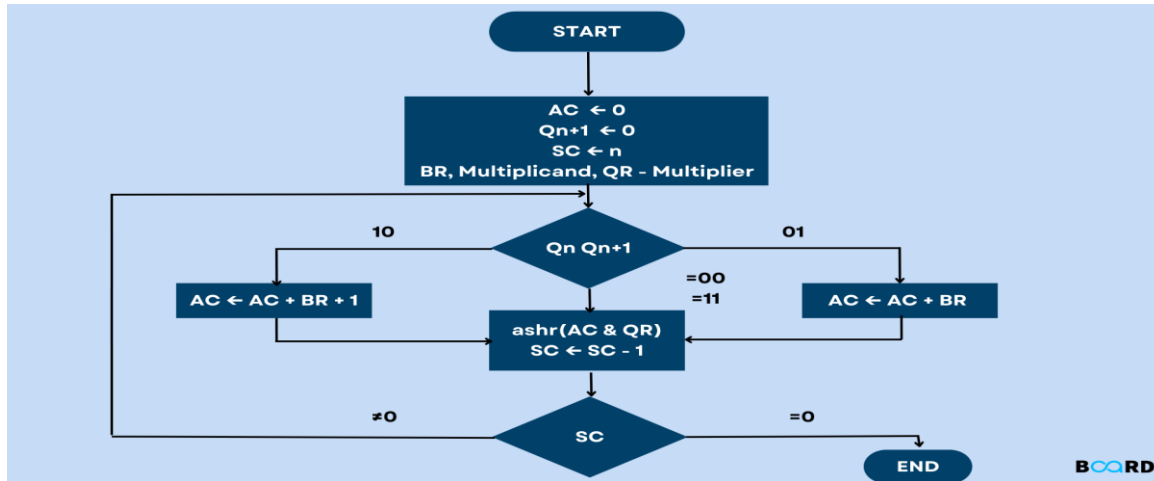
1. Faster multiplication than ordinary binary multiplication.
2. Supports both positive and negative numbers.
3. Reduces the number of addition and subtraction operations.
4. Efficient hardware implementation.
5. Widely used in processors and ALUs.

Disadvantages

1. More complex than normal multiplication.
2. Requires additional registers.
3. Difficult for beginners to understand.
4. Control logic is comparatively complex.

Applications

1. Arithmetic Logic Unit (ALU)
2. Microprocessors
3. Digital Signal Processors (DSP)
4. Embedded Systems
5. Computer Arithmetic Units



Division Operation (Restoring Division Algorithm)

Definition

Restoring Division Algorithm is a binary division algorithm used by computers to divide two binary numbers. It is called **Restoring Division** because whenever the subtraction produces a negative result, the previous value of the accumulator is **restored** before proceeding to the next step.

The algorithm uses the following registers:

- **A (Accumulator)** – Stores the partial remainder.
- **Q (Dividend Register)** – Stores the dividend and later the quotient.
- **M (Divisor Register)** – Stores the divisor.
- **n** – Number of bits.

Algorithm

1. Initialize **A = 0000**.
2. Load the Dividend into **Q**.
3. Load the Divisor into **M**.
4. Left shift the pair **AQ**.
5. Perform **A = A - M**.
6. If **A** is positive (**A ≥ 0**):
 - Set **Q₀ = 1**.
7. If **A** is negative (**A < 0**):

- Restore **A = A + M**.
- Set **Q₀ = 0**.

8. Repeat the above steps for **n** bits.

9. After completion:

- **Q = Quotient**
- **A = Remainder**

Solved Example

Divide 13 ÷ 3

Step 1: Binary Representation

Dividend (**Q**) = **1101** (13)

Divisor (**M**) = **0011** (3)

Accumulator (**A**) = **0000**

Number of bits = **4**

Initial Values

Register Value

A	0000
Q	1101
M	0011

Iteration Table

Step	Left Shift (AQ)	A = A – M	Q ₀	Action
Initial	A=0000 Q=1101	-	-	Initial Values
1	A=0001 Q=1010	0001-0011 = 1110 (Negative)	0	Restore A = 0001
2	A=0011 Q=0100	0011-0011 = 0000	1	Keep A
3	A=0000 Q=1001	0000-0011 = 1101 (Negative)	0	Restore A = 0000
4	A=0001 Q=0010	0001-0011 = 1110 (Negative)	0	Restore A = 0001

Final Result

Register	Value
Quotient (Q)	0100
Remainder (A)	0001

Verification

- Quotient = $0100_2 = 4_{10}$
- Remainder = $0001_2 = 1_{10}$

Therefore,

$13 \div 3 = 4$, Remainder = 1

Advantages

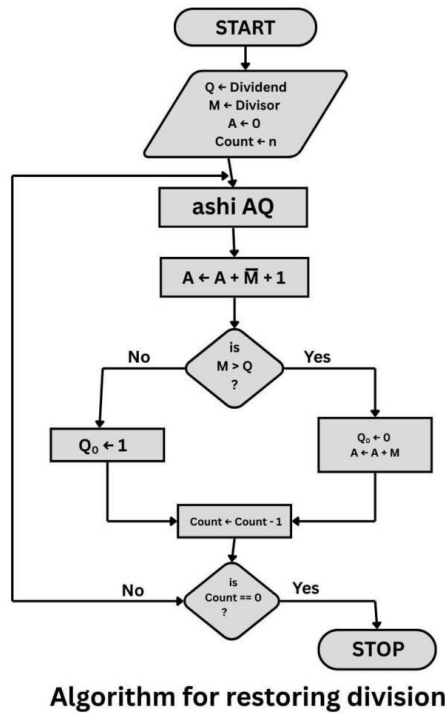
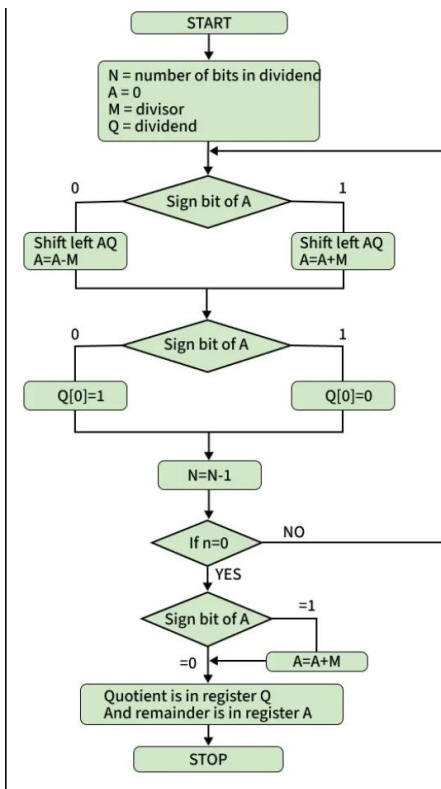
1. Easy to understand and implement.
2. Produces accurate quotient and remainder.
3. Suitable for hardware implementation.
4. Commonly used in digital systems.

Disadvantages

1. Slower than Non-Restoring Division.
2. Requires restoration when subtraction is negative.
3. More arithmetic operations increase execution time.

Applications

1. Arithmetic Logic Unit (ALU).
2. Microprocessors.
3. Digital Computers.
4. Embedded Systems.
5. Scientific and Engineering Calculations.



Non-Restoring Division Algorithm

Definition

Non-Restoring Division Algorithm is a binary division algorithm used in computers to divide two binary numbers. Unlike the **Restoring Division Algorithm**, this method **does not restore the previous value of the accumulator** when subtraction results in a negative value. Instead, the next operation (addition or subtraction) depends on whether the accumulator is positive or negative.

Because it avoids the restoration step, **Non-Restoring Division is faster and more efficient** than Restoring Division.

Registers Used

- **A (Accumulator):** Stores the partial remainder.
- **Q (Dividend Register):** Initially stores the dividend and finally stores the quotient.
- **M (Divisor Register):** Stores the divisor.
- **n:** Number of bits.

Algorithm

1. Initialize:
 - $A = 0000$
 - $Q = \text{Dividend}$

- $M = \text{Divisor}$
- 2. Left shift the combined register (A,Q).
- 3. If $A \geq 0$, perform $A = A - M$.
- 4. If $A < 0$, perform $A = A + M$.
- 5. If the new value of $A \geq 0$, set $Q_0 = 1$.
- 6. If the new value of $A < 0$, set $Q_0 = 0$.
- 7. Repeat the above steps n times.
- 8. If the final value of A is negative, restore it by $A = A + M$.
- 9. The final values are:
 - $Q = \text{Quotient}$
 - $A = \text{Remainder}$

Solved Example

Divide $13 \div 3$

Binary Representation

- Dividend (Q) = **1101**
- Divisor (M) = **0011**
- Accumulator (A) = **0000**

Initial Values

Register Value

A	0000
Q	1101
M	0011

Iteration Table

Step Left Shift (AQ) Operation A after Operation Q_0

Initial	0000 1101	—	0000	—
1	0001 1010	$A = A - M$	1110 (Negative)	0

Step Left Shift (AQ) Operation A after Operation Q₀

2	1101 0100	A = A + M 0000 (Positive)	1
3	0000 1001	A = A – M 1101 (Negative)	0
4	1011 0010	A = A + M 1110 (Negative)	0

Final Correction

Since the final value of **A** is negative:

A = A + M

1110 + 0011 = **0001**

Final Result

Output Value

Quotient **0100**

Remainder **0001**

Verification

- Quotient = **4**
- Remainder = **1**

Therefore,

13 ÷ 3 = 4, Remainder = 1

Advantages

1. Faster than Restoring Division.
 2. No restoration after every negative subtraction.
 3. Fewer arithmetic operations.
 4. Better processor performance.
 5. Efficient hardware implementation.
-

Disadvantages

1. Control logic is more complex.

- 2. Final correction may be required.
- 3. Difficult for beginners to understand.

Difference Between Restoring and Non-Restoring Division

Restoring Division	Non-Restoring Division
Restores accumulator when subtraction becomes negative.	Does not restore immediately.
Slower execution.	Faster execution.
More arithmetic operations.	Fewer arithmetic operations.
Simple control logic.	Complex control logic.
Lower efficiency.	Higher efficiency.

Applications

- 1. Arithmetic Logic Unit (ALU).
- 2. Microprocessors.
- 3. Digital Signal Processors (DSP).
- 4. Embedded Systems.
- 5. Scientific and Engineering Calculations.

Floating Point Arithmetic Operation

Definition

Floating Point Arithmetic Operation is the process of performing arithmetic operations such as **addition, subtraction, multiplication, and division** on floating-point numbers. It is used to represent and calculate **real (decimal) numbers** with high precision. Modern computers use the **IEEE 754 Standard** for floating-point representation and arithmetic.

A floating-point number consists of three parts:

- **Sign Bit (S):** Indicates whether the number is positive (0) or negative (1).
- **Exponent (E):** Represents the power of 2.
- **Mantissa (M) or Significand:** Represents the significant digits of the number.

IEEE 754 Single Precision Format

Field	Number of Bits	Description
Sign	1 Bit	Indicates the sign of the number

Field	Number of Bits	Description
Exponent	8 Bits	Stores the exponent with a bias of 127
Mantissa	23 Bits	Stores the fractional part of the number

Types of Floating Point Arithmetic Operations

1. Floating Point Addition

Definition:
Floating-point addition is performed by first aligning the exponents, then adding the mantissas, normalizing the result, and storing it in floating-point format.

Example:

$5.5 + 2.25 = 7.75$

Steps

- 1. Convert the numbers into binary.
- 2. Normalize both numbers.
- 3. Equalize the exponents.
- 4. Add the mantissas.
- 5. Normalize the result.
- 6. Round the result if required.

Example Table

Step	Result
Number 1	$5.5 = 1.011 \times 2^2$
Number 2	$2.25 = 1.001 \times 2^1$
Equalize Exponent	0.1001×2^2
Add Mantissas	1.1111
Final Result	7.75

2. Floating Point Subtraction

Definition:
Floating-point subtraction is similar to addition. The exponents are first aligned, then the mantissas are subtracted, and the result is normalized.

Example:

$7.5 - 2.5 = 5$

Steps

- 1. Convert both numbers into binary.
- 2. Normalize them.
- 3. Equalize the exponents.
- 4. Subtract the mantissas.
- 5. Normalize the result.
- 6. Store the final result.

Example Table

Step	Result
Number 1	$7.5 = 1.111 \times 2^2$
Number 2	$2.5 = 1.010 \times 2^1$
Equalize Exponent	0.101×2^2
Subtract Mantissas	1.010
Final Result	5

3. Floating Point Multiplication

Definition:
Floating-point multiplication is performed by multiplying the mantissas and adding the exponents. The result is then normalized.

Example:

$2.5 \times 1.5 = 3.75$

Steps

- 1. Convert the numbers into binary.
- 2. Normalize them.
- 3. Multiply the mantissas.
- 4. Add the exponents.
- 5. Normalize the result.

Example Table

Step	Result
Number 1	$2.5 = 1.01 \times 2^1$

Step	Result
Number 2	$1.5 = 1.1 \times 2^0$
Mantissa Multiplication	1.111
Exponent Addition	$1 + 0 = 1$
Final Result	3.75

4. Floating Point Division

Definition:

Floating-point division is performed by dividing the mantissas and subtracting the exponents. The result is then normalized.

Example:

$$6 \div 2 = 3$$

Steps

1. Convert both numbers into binary.
2. Normalize them.
3. Divide the mantissas.
4. Subtract the exponents.
5. Normalize the result.

Example Table

Step	Result
Number 1	$6 = 1.10 \times 2^2$
Number 2	$2 = 1.0 \times 2^1$
Mantissa Division	1.10
Exponent Subtraction	$2 - 1 = 1$
Final Result	3

Advantages

1. Represents very large and very small numbers efficiently.
2. Provides high precision for calculations.
3. Supports scientific and engineering computations.
4. Follows the IEEE 754 international standard.
5. Widely used in modern processors and computers.

Disadvantages

1. More complex than integer arithmetic.
2. Rounding errors may occur.
3. Requires additional hardware.
4. Slower than fixed-point arithmetic.
5. Overflow and underflow may occur.

Applications

1. Scientific Computing.
2. Engineering Simulations.
3. Artificial Intelligence.
4. Machine Learning.
5. Computer Graphics.
6. Weather Forecasting.
7. Financial and Banking Software.
8. Digital Signal Processing (DSP).

Design of Arithmetic Unit

Definition

The Arithmetic Unit (AU) is a major part of the Arithmetic Logic Unit (ALU) that performs arithmetic operations on binary data. It is designed using binary adders, multiplexers, registers, and logic circuits to execute operations such as addition, subtraction, increment, decrement, and data transfer.

The Arithmetic Unit receives input data from CPU registers, performs the required arithmetic operation based on the control signals, and stores the result in the destination register.

Components of Arithmetic Unit

1. Registers (A and B Registers)

Definition:

Registers are high-speed storage units that temporarily store binary operands before arithmetic operations are performed.

Function:

- **Store input operands.**
- **Transfer data to the Arithmetic Unit.**
- **Store intermediate results.**

2. Parallel Adder

Definition:

A Parallel Adder is a combinational circuit that performs binary addition of two n-bit binary numbers simultaneously.

Function:

- **Performs binary addition.**
- **Used in subtraction through Two's Complement.**
- **Generates Sum and Carry.**

3. Multiplexer (MUX)

Definition:

A Multiplexer (MUX) is a digital circuit that selects one input from multiple inputs according to the control signals.

Function:

- **Selects the required operand.**
- **Controls arithmetic operations.**
- **Transfers selected data to the Parallel Adder.**

4. Control Circuit

Definition:

The Control Circuit generates control signals that determine which arithmetic operation will be performed.

Function:

- **Selects arithmetic operation.**
- **Controls Multiplexer.**
- **Controls Carry Input.**

5. Carry Input (Cin)

Definition:

Carry Input is an additional input provided to the Parallel Adder for operations such as increment and subtraction.

Function:

- **Supports addition.**
- **Used in Two's Complement subtraction.**
- **Performs increment operation.**

Working of Arithmetic Unit

1. Binary operands are loaded into the registers.
2. The Control Unit generates the required control signals.
3. The Multiplexer selects the desired input.
4. The Parallel Adder performs the selected arithmetic operation.
5. The result is stored in the destination register.

Arithmetic Operations Performed

Control Input	Operation	Description
000	Transfer A	Output = A
001	Increment A	$A + 1$
010	Addition	$A + B$
011	Addition with Carry	$A + B + 1$
100	Addition with 1's Complement of B	$A + B'$
101	Subtraction	$A - B$ (using Two's Complement)
110	Decrement A	$A - 1$
111	Transfer A	Output remains A

Example

Suppose:

$A = 0110$ (6)

$B = 0011$ (3)

Control Input = 010 (Addition)

Operation:

```

  0110
+ 0011
-----
  1001

```

Result = $1001_2 = 9_{10}$

If the Control Input is 101 (Subtraction):

```

  0110
- 0011
-----
  0011

```

Result = $0011_2 = 3_{10}$

Advantages

1. Performs arithmetic operations at high speed.
2. Supports multiple arithmetic functions.
3. Reduces hardware complexity.
4. Easy integration with the ALU.
5. Improves CPU performance.

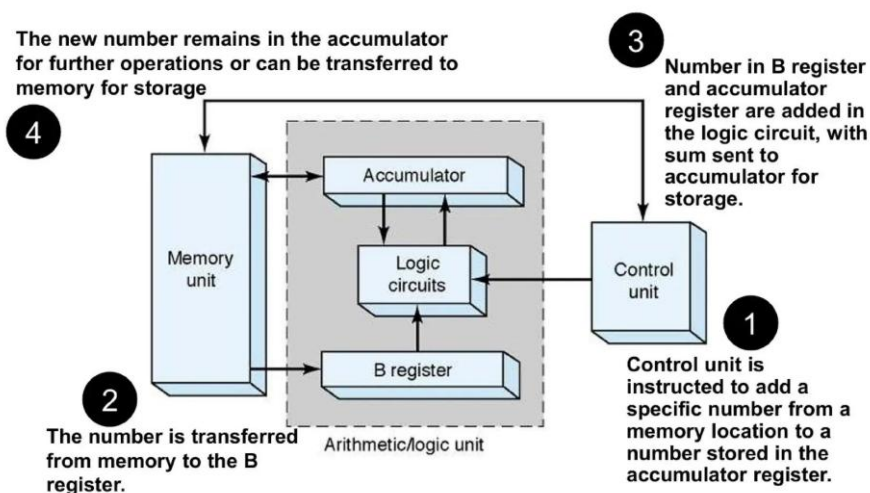
Disadvantages

1. Hardware becomes complex for large word sizes.
2. Overflow may occur during arithmetic operations.
3. Requires additional control circuitry.
4. More power consumption for high-speed designs.

Applications

1. Arithmetic Logic Unit (ALU).
2. Microprocessors.
3. Microcontrollers.
4. Digital Computers.
5. Embedded Systems.
6. Scientific and Engineering Calculations.
7. Digital Signal Processing (DSP).

Arithmetic Circuits



Introduction

Input/Output (I/O) Organization is the part of Computer Organization and Architecture (COA) that manages the communication between the CPU, Memory, and Input/Output devices. It provides an efficient method for transferring data between the computer system and external devices such as the keyboard, mouse, monitor, printer, hard disk, scanner, and network devices.

Since I/O devices operate at different speeds compared to the CPU, an I/O Organization is required to coordinate and control data transfer efficiently.

Objectives of I/O Organization

1. To provide communication between CPU and I/O devices.
2. To manage data transfer efficiently.
3. To reduce CPU waiting time.
4. To increase overall system performance.
5. To provide a standard interface for different peripheral devices.

Basic Components of I/O Organization

1. Central Processing Unit (CPU)

Definition:

The CPU is the brain of the computer that controls all operations. It sends commands to I/O devices and processes the received data.

Functions

- Controls I/O operations.
- Executes instructions.
- Communicates with memory and I/O devices.

2. Main Memory

Definition:

Main Memory temporarily stores programs and data while they are being processed. It acts as an intermediate storage area during I/O operations.

Functions

- Stores input data.
- Stores processed results.
- Exchanges data with the CPU and I/O devices.

3. I/O Interface (I/O Controller)

Definition:

An I/O Interface (or I/O Controller) is a hardware component that connects peripheral devices to the CPU and memory. It translates signals between the CPU and external devices.

Functions

- **Controls data transfer.**
- **Converts data formats.**
- **Synchronizes communication between the CPU and devices.**

4. Input Devices**Definition:**

Input devices are hardware devices used to enter data and instructions into the computer.

Examples

- **Keyboard**
- **Mouse**
- **Scanner**
- **Microphone**
- **Webcam**

5. Output Devices**Definition:**

Output devices are hardware devices that display or produce the processed information.

Examples

- **Monitor**
- **Printer**
- **Speaker**
- **Projector**
- **Plotter**

Working of I/O Organization

1. **The user enters data through an input device.**
2. **The input device sends the data to the I/O Interface.**
3. **The I/O Interface transfers the data to the Main Memory.**
4. **The CPU processes the data.**
5. **The processed result is stored in memory.**
6. **The output device receives the result through the I/O Interface and displays it to the user.**

Advantages

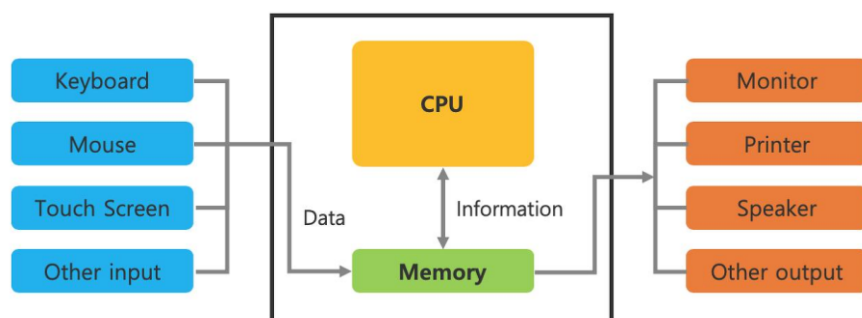
1. Enables communication between the CPU and peripheral devices.
2. Improves overall system performance.
3. Supports efficient data transfer.
4. Reduces CPU idle time.
5. Allows multiple devices to communicate with the computer.

Disadvantages

1. High-speed I/O hardware increases system cost.
2. Slow I/O devices can reduce overall performance.
3. Complex hardware design is required.
4. Data synchronization issues may occur.

Applications

1. Personal Computers.
2. Laptops.
3. Embedded Systems.
4. Banking Systems.
5. Industrial Automation.
6. Medical Equipment.
7. Communication Systems.
8. Network Servers.



I/O Interface – PCI Bus, SCSI Bus, USB

I/O Interface

Definition

An Input/Output (I/O) Interface is a hardware component that acts as a communication bridge between the CPU, Main Memory, and I/O devices. It controls the transfer of data and ensures

that devices with different speeds and data formats can communicate efficiently with the computer system.

The I/O Interface converts signals, manages data transfer, and synchronizes communication between the processor and peripheral devices.

Functions of I/O Interface

- 1. Provides communication between the CPU and peripheral devices.**
- 2. Controls the transfer of data between devices and memory.**
- 3. Converts data into a suitable format.**
- 4. Synchronizes the speed of the CPU and I/O devices.**
- 5. Detects and reports transmission errors.**
- 6. Generates control and status signals.**

Advantages

- 1. Provides efficient communication between hardware components.**
- 2. Supports multiple I/O devices.**
- 3. Reduces CPU workload.**
- 4. Improves overall system performance.**
- 5. Increases data transfer reliability.**

Disadvantages

- 1. Increases hardware complexity.**
- 2. Adds extra implementation cost.**
- 3. Requires compatibility between devices.**
- 4. May become a bottleneck if the interface speed is low.**

PCI Bus (Peripheral Component Interconnect Bus)

Definition

The PCI (Peripheral Component Interconnect) Bus is a high-speed parallel communication bus used to connect peripheral devices such as network cards, sound cards, graphics cards, and storage controllers to the motherboard. It was developed by Intel and became one of the most widely used expansion buses in desktop computers.

Features

- 1. High-speed parallel data transfer.**
- 2. Supports Plug and Play devices.**
- 3. Allows multiple expansion cards.**

4. Standard interface for peripheral devices.
5. Provides direct communication with the CPU and memory.

Advantages

1. High data transfer speed.
2. Easy installation of expansion cards.
3. Supports multiple devices simultaneously.
4. Reliable and standardized interface.

Disadvantages

1. Older technology compared to PCI Express (PCIe).
2. Limited bandwidth.
3. Shared bus architecture reduces performance when many devices are connected.

Applications

1. Sound Cards.
2. Network Interface Cards (NIC).
3. TV Tuner Cards.
4. Storage Controllers.
5. Expansion Cards.

SCSI Bus (Small Computer System Interface)

Definition

The SCSI (Small Computer System Interface) Bus is a high-performance interface standard used to connect multiple peripheral devices such as hard disks, scanners, printers, tape drives, and optical drives to a computer. It is mainly used in servers and professional workstations where high performance and reliability are required.

Features

1. Supports multiple devices on a single bus.
2. High-speed data transfer.
3. Supports internal and external devices.
4. Reliable communication.
5. Allows simultaneous operations.

Advantages

1. High performance.

2. Supports many devices on one bus.
3. Reliable for enterprise systems.
4. Suitable for multitasking environments.

Disadvantages

1. Expensive compared to USB.
2. Complex installation.
3. Requires proper device termination.
4. Rarely used in modern personal computers.

Applications

1. Servers.
2. Workstations.
3. RAID Storage Systems.
4. Tape Backup Devices.
5. High-performance Disk Drives.

USB (Universal Serial Bus)

Definition

The USB (Universal Serial Bus) is the most commonly used serial interface standard for connecting external devices to a computer. It supports Plug and Play and Hot Swapping, allowing devices to be connected or removed without restarting the computer.

Features

1. Plug and Play support.
2. Hot Swapping capability.
3. High-speed serial communication.
4. Provides both data transfer and power supply.
5. Supports a wide variety of devices.

Advantages

1. Easy to use.
2. Low cost.
3. High compatibility.
4. Supports many peripheral devices.
5. Portable and reliable.

Disadvantages

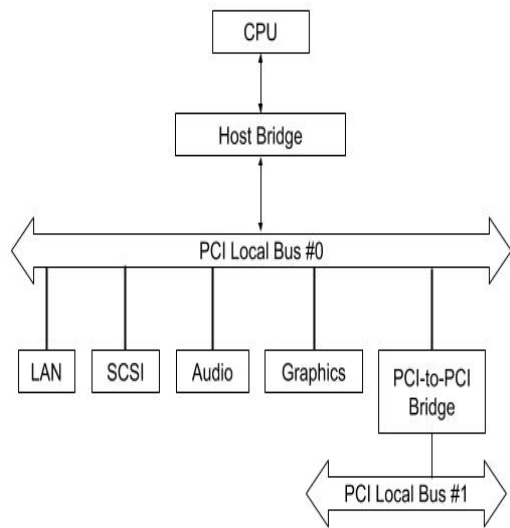
1. Lower speed than some specialized interfaces.
2. Cable length is limited.
3. Performance depends on the USB version.

Applications

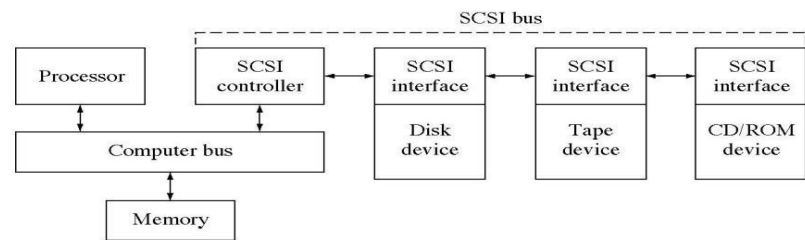
1. Keyboard.
2. Mouse.
3. Printer.
4. Scanner.
5. Pen Drive.
6. External Hard Disk.
7. Webcam.
8. Mobile Phone Connection.

Difference Between PCI Bus, SCSI Bus and USB

Feature	PCI Bus	SCSI Bus	USB
Full Form	Peripheral Component Interconnect	Small Computer System Interface	Universal Serial Bus
Type	Parallel Bus	High-Speed Peripheral Bus	Serial Bus
Connection	Internal Devices	Internal and External Devices	External Devices
Speed	High	Very High	Depends on USB Version
Cost	Medium	High	Low
Hot Swapping	No	Limited	Yes
Plug and Play	Yes	Limited	Yes
Common Applications	Graphics Card, Sound Card	Servers, Storage Devices	Keyboard, Mouse, Printer, Pen Drive



SCSI architecture



6/9/2014

Crowley OS Chap. 14

37

Data Transfer

Definition

Data Transfer is the process of moving data between the CPU, Main Memory, and Input/Output (I/O) devices. Since different devices operate at different speeds, various data transfer techniques are used to ensure efficient and reliable communication.

The main modes of data transfer are Serial Transfer, Parallel Transfer, Synchronous Transfer, Asynchronous Transfer, Direct Memory Access (DMA), and I/O Processor (IOP).

Types of Data Transfer

1. Serial Data Transfer

Definition

Serial Data Transfer is a method in which one bit of data is transmitted at a time over a single communication line. It is suitable for long-distance communication because it requires fewer wires and has lower transmission costs.

Features

1. Transfers one bit at a time.
2. Uses a single communication channel.
3. Suitable for long-distance communication.
4. Low hardware cost.

Example

USB, RS-232, Bluetooth, UART.

Advantages

1. Low cost.
2. Simple wiring.
3. Reliable over long distances.

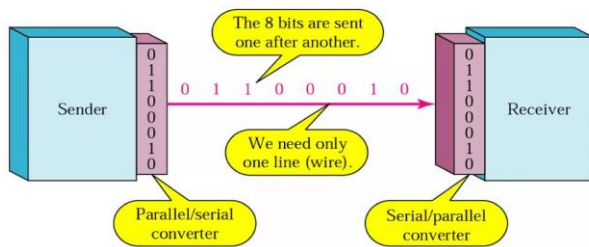
4. Less signal interference.

Disadvantages

1. Slower than parallel transfer.
2. Not suitable for high-speed bulk data transfer.

Diagram

Serial Transmission



Parallel Data Transfer

Definition

Parallel Data Transfer is a method in which multiple bits are transmitted simultaneously using multiple communication lines. It provides faster data transfer than serial communication but is generally suitable for short distances.

Features

1. Transfers multiple bits simultaneously.
2. Uses multiple data lines.
3. High-speed communication.
4. Suitable for short-distance transmission.

Example

Printer Port, Internal Data Bus.

Advantages

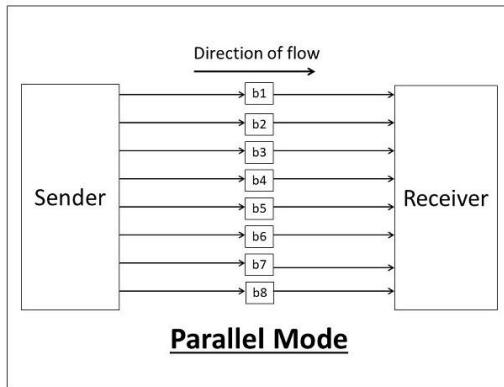
1. High data transfer speed.
2. Faster than serial communication.
3. Suitable for CPU-memory communication.

Disadvantages

1. Requires more wires.

2. Higher cost.
3. Signal interference increases with distance.

Diagram



Difference Between Serial and Parallel Data Transfer

Serial Transfer

Transfers one bit at a time.

Uses one communication line.

Suitable for long distances.

Lower cost.

Slower speed.

Parallel Transfer

Transfers multiple bits simultaneously.

Uses multiple communication lines.

Suitable for short distances.

Higher cost.

Faster speed.

Synchronous Data Transfer

Definition

Synchronous Data Transfer is a method in which the sender and receiver communicate using a common clock signal. Since both devices are synchronized by the same clock, data is transferred continuously without start and stop bits.

Features

1. Uses a common clock signal.
2. High-speed communication.
3. Continuous data transfer.
4. More efficient than asynchronous transfer.

Example

CPU to Memory communication.

Advantages

1. High speed.

2. Better efficiency.
3. Reliable communication.
4. Suitable for high-performance systems.

Disadvantages

1. Requires clock synchronization.
2. Hardware implementation is more complex.

Asynchronous Data Transfer

Definition

Asynchronous Data Transfer is a method in which the sender and receiver do not share a common clock signal. Each character is transmitted independently using start and stop bits.

Features

1. No common clock signal.
2. Uses start and stop bits.
3. Simple communication method.
4. Suitable for low-speed devices.

Example

Keyboard communication, Serial Port (RS-232).

Advantages

1. Simple implementation.
2. Low hardware cost.
3. No synchronization required.

Disadvantages

1. Slower due to start and stop bits.
2. Less efficient than synchronous transfer.

Difference Between Synchronous and Asynchronous Data Transfer

Synchronous Transfer	Asynchronous Transfer
----------------------	-----------------------

Uses a common clock signal.	Does not use a common clock signal.
-----------------------------	-------------------------------------

High-speed communication.	Lower speed.
---------------------------	--------------

No start and stop bits.	Uses start and stop bits.
-------------------------	---------------------------

More efficient.	Less efficient.
-----------------	-----------------

Synchronous Transfer

Asynchronous Transfer

Hardware is more complex. Hardware is simpler.

Direct Memory Access (DMA)

Definition

Direct Memory Access (DMA) is a data transfer technique that allows I/O devices to transfer data directly to or from the main memory without continuous involvement of the CPU. A special hardware unit called the DMA Controller manages the transfer, allowing the CPU to perform other tasks simultaneously.

Working

- 1. CPU initializes the DMA Controller.**
- 2. DMA Controller takes control of the system bus.**
- 3. Data is transferred directly between the I/O device and memory.**
- 4. DMA Controller sends an interrupt to the CPU after completion.**

Advantages

- 1. High-speed data transfer.**
- 2. Reduces CPU workload.**
- 3. Improves overall system performance.**
- 4. Efficient for large data transfers.**

Disadvantages

- 1. Requires additional hardware.**
- 2. Hardware design is more complex.**
- 3. Bus conflicts may occur.**

Applications

- 1. Hard Disk Drives.**
- 2. SSDs.**
- 3. Network Cards.**
- 4. Audio and Video Streaming.**
- 5. High-Speed Data Acquisition Systems.**

I/O Processor (IOP)

Definition

An I/O Processor (IOP) is a specialized processor designed to control and manage Input/Output operations independently of the CPU. It executes I/O instructions, controls peripheral devices, and transfers data between memory and I/O devices, thereby reducing the CPU's workload.

Functions

- 1. Controls I/O devices.**
- 2. Manages data transfer.**
- 3. Executes I/O instructions.**
- 4. Reduces CPU involvement.**
- 5. Improves system efficiency.**

Advantages

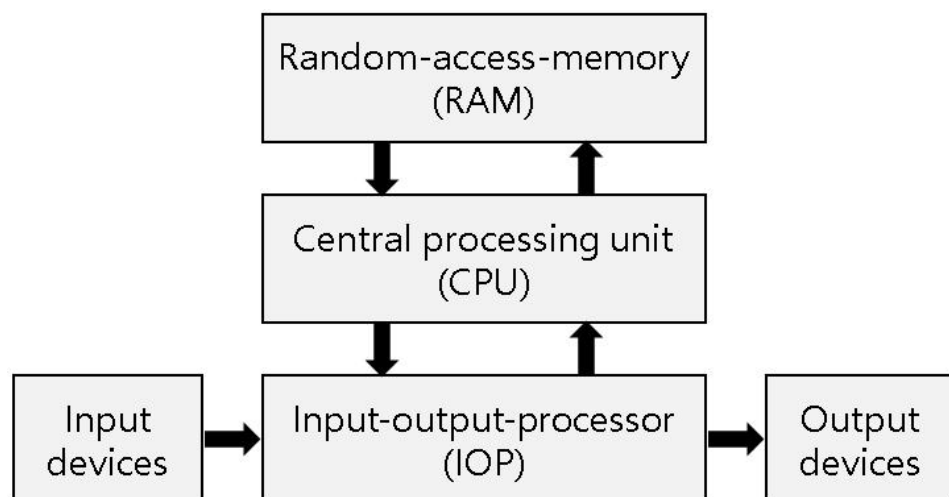
- 1. Reduces CPU workload.**
- 2. Supports multiple I/O operations simultaneously.**
- 3. Increases overall system performance.**
- 4. Improves multitasking capability.**

Disadvantages

- 1. Additional hardware cost.**
- 2. Complex design.**
- 3. Higher power consumption.**

Applications

- 1. Mainframe Computers.**
- 2. Servers.**
- 3. High-Performance Computing Systems.**
- 4. Industrial Automation.**
- 5. Network Systems.**



Unit IV: Memory Organization

Introduction

Memory Organization refers to the arrangement and management of different types of memory in a computer system. It defines how data, instructions, and programs are stored, accessed, and transferred between the CPU, Main Memory, and Secondary Memory.

Memory organization plays a vital role in improving the speed, efficiency, and overall performance of a computer system. The CPU stores programs and data in memory before processing them.

Objectives of Memory Organization

- 1. To store data and instructions efficiently.**
- 2. To provide fast access to information.**
- 3. To improve CPU performance.**
- 4. To reduce memory access time.**
- 5. To ensure reliable storage and retrieval of data.**

Types of Memory

- 1. Primary (Main) Memory**
- 2. Secondary Memory**

In this unit, we mainly study Main Memory, which consists of RAM and ROM.

Main Memory

Definition

Main Memory, also known as **Primary Memory** or **Internal Memory**, is the memory directly accessed by the CPU. It stores the programs, data, and instructions currently being processed. Main memory provides fast access compared to secondary storage but has limited capacity.

Main memory consists of two main types:

- 1. RAM (Random Access Memory)**
- 2. ROM (Read Only Memory)**

Characteristics of Main Memory

- 1. Directly accessible by the CPU.**
- 2. High-speed memory.**
- 3. Stores currently running programs.**
- 4. Limited storage capacity.**
- 5. Essential for system operation.**

Advantages

- 1. Fast data access.**
- 2. Improves processing speed.**
- 3. Supports multitasking.**
- 4. Direct communication with the CPU.**

Disadvantages

- 1. Limited storage capacity.**
- 2. More expensive than secondary memory.**
- 3. RAM loses data when power is turned off.**

RAM (Random Access Memory)

Definition

Random Access Memory (RAM) is a volatile primary memory that temporarily stores data, programs, and instructions currently being executed by the CPU. Since RAM is volatile, all stored data is lost when the power supply is switched off.

RAM allows the CPU to read and write data quickly, making it one of the fastest memories in a computer.

Characteristics of RAM

- 1. Volatile memory.**
- 2. Read and write memory.**
- 3. High-speed access.**
- 4. Temporary storage.**
- 5. Directly connected to the CPU.**

Types of RAM

1. SRAM (Static RAM)

Definition

Static Random Access Memory (SRAM) stores data using flip-flops and does not require refreshing as long as power is available.

Features

- Very fast.**
- Expensive.**
- Low storage capacity.**
- Used for cache memory.**

2. DRAM (Dynamic RAM)

Definition

Dynamic Random Access Memory (DRAM) stores data using capacitors and requires periodic refreshing to retain data.

Features

- **Slower than SRAM.**
- **Less expensive.**
- **Higher storage capacity.**
- **Used as main system memory.**

Advantages of RAM

- 1. Very high speed.**
- 2. Direct access by the CPU.**
- 3. Supports multitasking.**
- 4. Improves overall computer performance.**
- 5. Allows both reading and writing operations.**

Disadvantages of RAM

- 1. Volatile memory.**
- 2. Data is lost when power is off.**
- 3. Higher cost than secondary storage.**
- 4. Limited storage capacity.**

Applications

- 1. Running operating systems.**
- 2. Executing application programs.**
- 3. Temporary storage during processing.**
- 4. Gaming and multimedia applications.**
- 5. Scientific computing.**

ROM (Read Only Memory)

Definition

Read Only Memory (ROM) is a non-volatile primary memory used to permanently store instructions required for booting the computer. The data stored in ROM is retained even when the power is turned off.

ROM mainly stores the BIOS (Basic Input/Output System) or firmware, which initializes the hardware during system startup.

Characteristics of ROM

- 1. Non-volatile memory.**
- 2. Permanent storage.**
- 3. Mostly read-only.**
- 4. Stores firmware and boot programs.**
- 5. Does not lose data after power failure.**

Types of ROM

1. PROM (Programmable Read Only Memory)

Can be programmed only once by the user.

2. EPROM (Erasable Programmable Read Only Memory)

Can be erased using ultraviolet (UV) light and reprogrammed.

3. EEPROM (Electrically Erasable Programmable Read Only Memory)

Can be erased and programmed electrically without removing it from the computer.

4. Flash ROM

A modern type of EEPROM that can be erased and rewritten in blocks. It is widely used in BIOS chips, SSDs, USB flash drives, and memory cards.

Advantages of ROM

- 1. Non-volatile memory.**
- 2. Permanent storage.**
- 3. Reliable and secure.**
- 4. No refresh operation required.**
- 5. Stores important system programs.**

Disadvantages of ROM

- 1. Slower than RAM.**
- 2. Difficult to modify stored data.**
- 3. Limited writing capability.**
- 4. Less flexible than RAM.**

Applications

- 1. BIOS/Firmware.**

2. Embedded Systems.
3. Mobile Phones.
4. Printers.
5. Consumer Electronics.
6. Automotive Systems.

Difference Between RAM and ROM

RAM	ROM
Random Access Memory	Read Only Memory
Volatile memory	Non-volatile memory
Data is lost when power is off	Data is retained after power is off
Read and write operations	Mostly read-only
Temporary storage	Permanent storage
Faster	Slower
Used during program execution	Used for booting and firmware

Secondary Memory

Definition

Secondary Memory is a non-volatile memory used for the permanent storage of data, programs, and files. Unlike Main Memory (RAM), data stored in secondary memory is not lost when the power is turned off. It provides large storage capacity at a lower cost but has slower access speed than primary memory.

Secondary memory is also known as Auxiliary Memory or External Memory.

Characteristics of Secondary Memory

- 1. Non-volatile memory.**
- 2. Permanent storage of data.**
- 3. Large storage capacity.**
- 4. Slower than primary memory.**
- 5. Less expensive than RAM.**
- 6. Used to store operating systems, applications, documents, videos, and multimedia files.**

Advantages

1. Provides permanent storage.
2. Large storage capacity.
3. Cost-effective compared to primary memory.
4. Data is retained even after power failure.
5. Suitable for backup and long-term storage.

Disadvantages

1. Slower than primary memory.
2. Cannot be accessed directly by the CPU.
3. Requires more access time.
4. Mechanical storage devices may wear out over time.

Applications

1. Storing Operating Systems.
2. Software Installation.
3. Data Backup.
4. Multimedia Storage.
5. Database Management.
6. Cloud Backup Systems.

Magnetic Tape

Definition

Magnetic Tape is a secondary storage device that stores data on a plastic tape coated with magnetic material. Data is stored sequentially, meaning it is accessed in the order in which it is recorded. It is mainly used for backup and archival purposes because of its high storage capacity and low cost.

Characteristics

1. Sequential Access Storage.
2. Large storage capacity.
3. Low cost.
4. Portable storage medium.
5. Suitable for long-term data backup.

Advantages

1. Very low storage cost.

2. High storage capacity.
3. Suitable for backup and archiving.
4. Long data retention period.

Disadvantages

1. Slow data access.
2. Sequential access only.
3. Not suitable for frequent data retrieval.
4. Mechanical wear over time.

Applications

1. Data Backup.
2. Government Records.
3. Banking Systems.
4. Scientific Data Storage.
5. Cloud Data Archiving.

Magnetic Disk

Definition

A Magnetic Disk is a secondary storage device that stores data on rotating circular disks coated with magnetic material. Data can be accessed directly using read/write heads, making it much faster than magnetic tape.

Examples include Hard Disk Drives (HDDs) and Floppy Disks.

Characteristics

1. Direct (Random) Access Storage.
2. High storage capacity.
3. Faster than magnetic tape.
4. Reusable storage device.
5. Supports both read and write operations.

Advantages

1. Fast data access.
2. Large storage capacity.
3. Supports random access.
4. Reliable for everyday use.

5. Lower cost than SSDs.

Disadvantages

1. Mechanical parts may fail.
2. Slower than SSDs.
3. Sensitive to physical shocks.
4. Higher power consumption than solid-state storage.

Applications

1. Personal Computers.
2. Laptops.
3. Servers.
4. Database Storage.
5. Multimedia Storage.

Optical Storage

Definition

Optical Storage is a secondary storage technology in which data is stored and read using a laser beam. It is widely used for storing software, multimedia files, music, movies, and backups.

Common examples include CD (Compact Disc), DVD (Digital Versatile Disc), and Blu-ray Disc.

Types of Optical Storage

1. CD (Compact Disc)

- Storage Capacity: 700 MB
- Used for music, software, and documents.

2. DVD (Digital Versatile Disc)

- Storage Capacity: 4.7 GB to 8.5 GB
- Used for movies, software installation, and data storage.

3. Blu-ray Disc

- Storage Capacity: 25 GB to 50 GB (or more)
- Used for high-definition video and large data storage.

Characteristics

1. Laser-based storage technology.
2. Portable and lightweight.
3. Long data life.

- 4. Resistant to magnetic fields.
- 5. Available in different storage capacities.

Advantages

- 1. Low cost.
- 2. Portable.
- 3. Long lifespan.
- 4. Resistant to magnetic interference.
- 5. Easy to store and transport.

Disadvantages

- 1. Slower than HDDs and SSDs.
- 2. Easily scratched or damaged.
- 3. Limited rewrite capability (depending on disc type).
- 4. Optical drives are becoming less common in modern computers.

Applications

- 1. Software Distribution.
- 2. Movie Storage.
- 3. Music Albums.
- 4. Educational Content.
- 5. Data Backup.
- 6. Medical Data Storage.

Difference Between Magnetic Tape, Magnetic Disk and Optical Storage

Feature	Magnetic Tape	Magnetic Disk	Optical Storage
Access Method	Sequential Access	Random Access	Random Access
Speed	Slow	Fast	Medium
Storage Medium	Magnetic Tape	Magnetic Disk	Optical Disc
Storage Capacity	Very High	High	Medium to High
Cost	Low	Medium	Low
Portability	High	Moderate	High
Reusability	Yes	Yes	Depends on Disc Type

Feature	Magnetic Tape	Magnetic Disk	Optical Storage
Common Uses	Backup & Archive	Daily Storage	Software, Movies, Music

Important Exam Points

- **Secondary Memory is non-volatile and stores data permanently.**
- **Magnetic Tape uses sequential access, making it suitable for backups and archives.**
- **Magnetic Disk uses random access, providing faster data retrieval than magnetic tape.**
- **Optical Storage uses laser technology and includes CD, DVD, and Blu-ray Disc.**
- **Magnetic Disk is commonly used for everyday storage, while Magnetic Tape is preferred for long-term backups and Optical Storage is widely used for software and multimedia distribution.**

Cache Memory

Definition

Cache Memory is a small, high-speed memory located between the CPU and Main Memory (RAM). It stores frequently used data and instructions so that the CPU can access them quickly. Since cache memory is much faster than RAM, it significantly improves the overall performance of the computer.

Cache memory is generally built using SRAM (Static Random Access Memory) because of its high speed.

Characteristics of Cache Memory

- 1. Very high-speed memory.**
- 2. Located between CPU and Main Memory.**
- 3. Stores frequently accessed data.**
- 4. Small storage capacity.**
- 5. Built using SRAM.**
- 6. Reduces memory access time.**

Working of Cache Memory

- 1. CPU requests data.**
- 2. Cache memory checks whether the data is available.**
- 3. If the data is found, it is called a Cache Hit.**
- 4. If the data is not found, it is called a Cache Miss.**
- 5. In case of a cache miss, the data is fetched from RAM and stored in the cache for future use.**

Advantages

- 1. Very fast data access.**
- 2. Reduces CPU waiting time.**
- 3. Improves system performance.**
- 4. Reduces memory access time.**
- 5. Increases execution speed.**

Disadvantages

- 1. High cost.**
- 2. Limited storage capacity.**
- 3. Complex design.**
- 4. Uses more power than RAM.**

Applications

- 1. Microprocessors.**
- 2. Personal Computers.**
- 3. Laptops.**
- 4. Servers.**
- 5. Embedded Systems.**

Cache Structure and Design

Definition

Cache Structure refers to the internal organization of cache memory, while **Cache Design** refers to the techniques used to improve cache performance and efficiency.

A cache memory is divided into blocks (cache lines), each capable of storing one block of data from the main memory.

Components of Cache Structure

1. Data Array

Stores the actual data copied from the main memory.

2. Tag Memory

Stores the address (tag) corresponding to each cache block.

3. Valid Bit

Indicates whether the cache block contains valid data.

4. Control Logic

Controls searching, reading, writing, and updating of cache memory.

Advantages

- 1. Faster access to frequently used data.**
- 2. Reduces memory latency.**
- 3. Improves CPU efficiency.**

Disadvantages

- 1. Expensive hardware.**
- 2. Complex implementation.**

Cache Mapping Scheme

Definition

A Cache Mapping Scheme is the method used to determine where a block of main memory will be stored in the cache memory.

There are three mapping techniques.

1. Direct Mapping

Definition

In Direct Mapping, each block of the main memory can be stored in only one fixed cache location.

Advantages

- 1. Simple implementation.**
- 2. Low cost.**
- 3. Fast mapping.**

Disadvantages

- 1. High conflict misses.**
- 2. Lower cache utilization.**

2. Fully Associative Mapping

Definition

In Fully Associative Mapping, any block of the main memory can be placed in any cache block.

Advantages

- 1. Lowest conflict misses.**
- 2. Better cache utilization.**
- 3. Higher flexibility.**

Disadvantages

1. Expensive hardware.
2. Complex searching.
3. Slower implementation.

3. Set Associative Mapping

Definition

Set Associative Mapping is a combination of Direct Mapping and Fully Associative Mapping. The cache is divided into sets, and a memory block can be stored in any block within its assigned set.

Advantages

1. Reduces conflict misses.
2. Better performance.
3. Moderate cost.

Disadvantages

1. More complex than Direct Mapping.
2. Higher hardware cost.

Difference Between Cache Mapping Schemes

Direct Mapping	Fully Associative Mapping	Set Associative Mapping
Fixed cache location	Any cache location	Any location within a set
Simple	Complex	Moderate
Low cost	High cost	Medium cost
High conflict misses	Very low conflict misses	Moderate conflict misses
Fast	Slow	Medium

Cache Replacement Algorithm

Definition

A Cache Replacement Algorithm determines which cache block should be replaced when the cache becomes full and a new memory block needs to be loaded.

Types of Replacement Algorithms

1. FIFO (First In First Out)

The block that entered the cache first is replaced first.

Advantages

- Simple implementation.
- Low overhead.

Disadvantages

- May remove frequently used data.

2. LRU (Least Recently Used)

The block that has not been used for the longest time is replaced.

Advantages

- Higher cache hit ratio.
- Better performance.

Disadvantages

- Complex implementation.

3. Random Replacement

A cache block is selected randomly for replacement.

Advantages

- Very simple.
- Low implementation cost.

Disadvantages

- Performance is unpredictable.

Improving Cache Performance

Definition

Improving Cache Performance means increasing the Cache Hit Ratio and reducing the Cache Miss Ratio so that the CPU spends less time waiting for data.

Techniques

1. Increase Cache Size

Stores more data and reduces cache misses.

2. Multi-Level Cache

Uses L1, L2, and L3 caches for better performance.

3. Better Mapping Techniques

Using Set Associative Mapping instead of Direct Mapping.

4. Efficient Replacement Algorithms

Using LRU instead of FIFO when appropriate.

5. Prefetching

Loads frequently required data before the CPU requests it.

Advantages

- 1. Faster execution.**
- 2. Higher cache hit ratio.**
- 3. Better CPU utilization.**
- 4. Improved system performance.**

Virtual Memory

Definition

Virtual Memory is a memory management technique in which a part of the secondary memory (hard disk/SSD) is used as an extension of the main memory (RAM). It allows programs larger than the available RAM to execute.

Virtual memory increases the apparent memory size of a computer.

Working

- 1. CPU requests data.**
- 2. If the data is not available in RAM, a page fault occurs.**
- 3. The required page is copied from secondary memory to RAM.**
- 4. The CPU continues execution.**

Advantages

- 1. Executes large programs.**
- 2. Better memory utilization.**
- 3. Supports multitasking.**
- 4. Reduces RAM limitations.**

Disadvantages

- 1. Slower than RAM.**
- 2. Page faults reduce performance.**
- 3. Increased disk access time.**

Applications

- 1. Operating Systems.**
- 2. Large Applications.**
- 3. Multitasking Systems.**

4. Database Servers.

Memory Management Hardware

Definition

Memory Management Hardware consists of hardware components that control memory allocation, address translation, protection, and virtual memory operations. It ensures efficient and secure use of memory resources.

Components

1. Memory Management Unit (MMU)

Definition

The Memory Management Unit (MMU) is a hardware component that converts logical (virtual) addresses into physical addresses.

Functions

- 1. Address translation.**
- 2. Memory protection.**
- 3. Virtual memory support.**
- 4. Access control.**

2. Base Register

Stores the starting physical address of a program.

3. Limit Register

Stores the maximum size of the program and prevents illegal memory access.

4. Translation Lookaside Buffer (TLB)

A TLB is a small high-speed cache that stores recently used page table entries, reducing address translation time.

Advantages

- 1. Faster address translation.**
- 2. Better memory protection.**
- 3. Efficient virtual memory management.**
- 4. Supports multitasking.**
- 5. Improves system performance.**

Disadvantages

- 1. Increases hardware complexity.**
- 2. Additional implementation cost.**

3. More power consumption.

Difference Between Cache Memory and Virtual Memory

Cache Memory	Virtual Memory
High-speed memory between CPU and RAM	Uses secondary memory as an extension of RAM
Built using SRAM	Uses HDD/SSD
Improves CPU speed	Increases available memory
Very fast	Slower than RAM
Small capacity	Very large capacity

Important Exam Points

- Cache Memory is a high-speed SRAM placed between the CPU and RAM.
- The three cache mapping schemes are Direct Mapping, Fully Associative Mapping, and Set Associative Mapping.
- Common replacement algorithms are FIFO, LRU, and Random Replacement.
- Virtual Memory allows programs larger than RAM to execute by using secondary storage.
- The Memory Management Unit (MMU) translates logical addresses into physical addresses, while the TLB speeds up address translation by caching recent page table entries

Unit V: Multiprocessors

Introduction

A Multiprocessor System is a computer system that contains two or more processors (CPUs) connected to a common main memory and input/output devices. These processors work together to execute multiple tasks simultaneously, resulting in higher processing speed, improved reliability, and better system performance.

Multiprocessor systems are widely used in supercomputers, servers, cloud computing, scientific research, artificial intelligence, and large database systems.

Definition

A Multiprocessor is a computer system in which multiple processors share a common memory, input/output devices, and operating system. Each processor can execute instructions independently while cooperating with other processors to complete tasks efficiently.

Features of Multiprocessor System

1. Contains two or more processors.
2. Shared main memory.

3. Shared input/output devices.
4. Supports parallel processing.
5. High processing speed.
6. Better system reliability.
7. Efficient resource utilization.

Characteristics of Multiprocessor

Definition

Characteristics of Multiprocessor are the important features that describe how multiple processors work together to improve the performance and efficiency of a computer system.

Characteristics

1. Multiple Processors

A multiprocessor system consists of two or more processors working together to execute different tasks simultaneously.

2. Shared Memory

All processors access the same main memory, allowing them to exchange data efficiently.

3. Parallel Processing

Different processors execute different parts of a program at the same time, reducing execution time.

4. Increased Throughput

Multiple processors complete more tasks in less time, increasing the number of jobs processed per unit time.

5. High Reliability

If one processor fails, the remaining processors can continue executing tasks, improving system reliability.

6. Load Sharing

The workload is distributed among all processors, preventing any single processor from becoming overloaded.

7. Resource Sharing

Processors share hardware resources such as memory, storage devices, buses, and I/O devices.

8. Scalability

Additional processors can be added to improve system performance without redesigning the entire system.

Advantages of Multiprocessor

- 1. High processing speed.**
- 2. Improved system performance.**
- 3. Supports multitasking.**
- 4. Better resource utilization.**
- 5. Increased reliability.**
- 6. Reduced execution time.**
- 7. Higher throughput.**
- 8. Supports parallel computing.**

Disadvantages of Multiprocessor

- 1. High implementation cost.**
- 2. Complex hardware design.**
- 3. Difficult synchronization between processors.**
- 4. Complex operating system support.**
- 5. Increased power consumption.**

Applications

- 1. Supercomputers.**
- 2. Cloud Computing.**
- 3. Database Servers.**
- 4. Artificial Intelligence.**
- 5. Machine Learning.**
- 6. Scientific Simulations.**
- 7. Banking Systems.**
- 8. Weather Forecasting.**

Structure of Multiprocessor

Definition

The Structure of a Multiprocessor System refers to the arrangement and interconnection of multiple processors, shared memory, buses, and input/output devices. In a multiprocessor system, all processors communicate through a common communication system and access the same main memory.

Components of Multiprocessor Structure

1. Processor (CPU)

Definition

The processor executes instructions and performs arithmetic, logical, and control operations.

Functions

- 1. Executes instructions.**
- 2. Processes data.**
- 3. Communicates with memory.**
- 4. Controls system operations.**

2. Shared Main Memory

Definition

Shared Main Memory stores programs and data that can be accessed by all processors.

Functions

- 1. Stores instructions.**
- 2. Stores data.**
- 3. Provides common access to processors.**

3. System Bus

Definition

The System Bus connects processors, memory, and I/O devices and transfers data, addresses, and control signals.

Functions

- 1. Data transfer.**
- 2. Address transfer.**
- 3. Control signal transmission.**

4. Input/Output Devices

Definition

Input/Output devices allow communication between the computer system and the external environment.

Examples

- Keyboard**
- Mouse**
- Monitor**

- Printer
- Hard Disk

5. Bus Controller

Definition

The Bus Controller manages access to the shared bus and prevents conflicts when multiple processors request the bus simultaneously.

Functions

1. Bus arbitration.
2. Controls data transfer.
3. Prevents bus conflicts.

Working of Multiprocessor System

1. A program is divided into multiple tasks.
2. Each processor receives a different task.
3. All processors execute their tasks simultaneously.
4. Processors access the shared memory whenever required.
5. The results are combined to produce the final output.

Advantages of Multiprocessor Structure

1. Faster execution of programs.
2. Efficient memory sharing.
3. Supports parallel processing.
4. Better fault tolerance.
5. Higher system efficiency.

Disadvantages of Multiprocessor Structure

1. Complex synchronization.
2. High hardware cost.
3. Bus contention may occur.
4. Difficult operating system design.

Difference Between Single Processor and Multiprocessor

Single Processor	Multiprocessor
One CPU	Two or More CPUs

Single Processor

Multiprocessor

Lower processing speed Higher processing speed

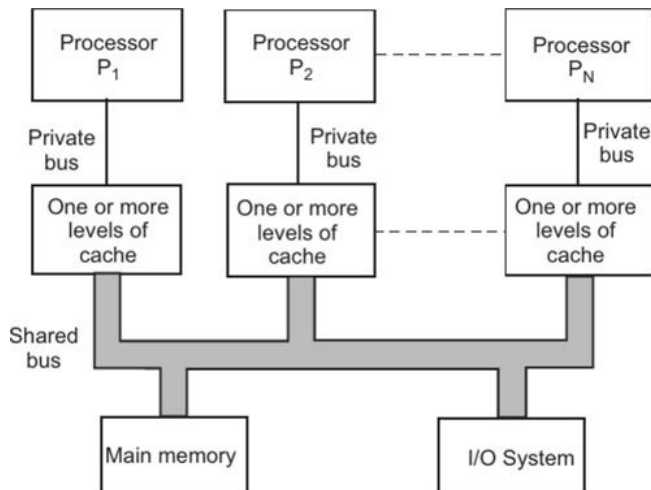
No parallel execution Supports parallel execution

Lower cost Higher cost

Lower reliability Higher reliability

Limited multitasking Better multitasking

Less resource sharing Shared resources among processors



Inter-Processor Arbitration

Definition

Inter-Processor Arbitration is the process of deciding which processor will gain access to a shared resource (such as the system bus or shared memory) when multiple processors request it at the same time. It ensures that only one processor uses the shared resource at a time, preventing conflicts and ensuring smooth system operation.

Need for Inter-Processor Arbitration

1. Prevents bus conflicts.
2. Avoids simultaneous access to shared memory.
3. Ensures fair allocation of resources.
4. Improves system efficiency.
5. Maintains data consistency.

Types of Arbitration

1. Centralized Arbitration

Definition

In Centralized Arbitration, a single hardware component called the Bus Arbiter decides which processor gets access to the shared bus.

Advantages

- 1. Simple implementation.**
- 2. Easy to control.**
- 3. Faster decision making.**

Disadvantages

- 1. Single point of failure.**
- 2. Bus arbiter may become a bottleneck.**

2. Distributed Arbitration

Definition

In Distributed Arbitration, there is no central controller. Each processor participates in the arbitration process and determines which processor gets bus access.

Advantages

- 1. No single point of failure.**
- 2. Better reliability.**
- 3. Suitable for large multiprocessor systems.**

Disadvantages

- 1. Complex implementation.**
- 2. Higher hardware cost.**

Applications

- 1. Multiprocessor Systems.**
- 2. Multi-core Processors.**
- 3. Servers.**
- 4. High-Performance Computing Systems.**

Important Points

- Prevents simultaneous bus access.**
- Ensures fair resource allocation.**
- Improves system performance.**

Inter-Processor Communication (IPC)

Definition

Inter-Processor Communication (IPC) is the process through which two or more processors exchange data, instructions, or control information while executing a program. It enables processors to cooperate and complete parallel tasks efficiently.

Methods of Inter-Processor Communication

1. Shared Memory Communication

Processors communicate by reading and writing data in a common shared memory.

2. Message Passing

Processors communicate by sending and receiving messages through communication channels.

Advantages

- 1. Fast communication.**
- 2. Efficient parallel processing.**
- 3. Better resource sharing.**
- 4. Increases system performance.**

Disadvantages

- 1. Synchronization problems.**
- 2. Communication overhead.**
- 3. Complex implementation.**

Applications

- 1. Parallel Computing.**
- 2. Cloud Computing.**
- 3. Distributed Systems.**
- 4. Scientific Simulations.**
- 5. Artificial Intelligence.**

Synchronization

Definition

Synchronization is the process of coordinating multiple processors so that they can access shared data and resources safely without causing conflicts or data inconsistency.

Synchronization ensures that only one processor accesses a critical resource at a particular time.

Need for Synchronization

- 1. Prevents data inconsistency.**
- 2. Avoids race conditions.**

3. Ensures correct execution of parallel programs.
4. Protects shared resources.
5. Maintains data integrity.

Synchronization Techniques

1. Lock (Mutex)

A processor locks a shared resource before using it. Other processors must wait until the lock is released.

2. Semaphore

A semaphore is a synchronization variable used to control access to shared resources by multiple processors.

3. Barrier Synchronization

All processors wait until every processor reaches a specific point before continuing execution.

Advantages

1. Prevents data corruption.
2. Ensures correct program execution.
3. Improves system reliability.
4. Supports safe resource sharing.

Disadvantages

1. Increased execution overhead.
2. Deadlock may occur.
3. Complex programming.
4. Reduced performance if synchronization is excessive.

Difference Between Inter-Processor Arbitration, Communication and Synchronization

Inter-Processor Arbitration	Inter-Processor Communication	Synchronization
Decides which processor gets access to a shared resource.	Exchanges data and information between processors.	Coordinates processors while accessing shared resources.
Prevents bus conflicts.	Enables cooperation among processors.	Prevents race conditions and data inconsistency.
Uses arbitration techniques.	Uses shared memory or message passing.	Uses locks, semaphores, and barriers.

Inter-Processor Arbitration	Inter-Processor Communication	Synchronization
Focuses on resource allocation.	Focuses on data exchange.	Focuses on safe execution of parallel tasks.

Advantages of These Techniques

- 1. Improve overall multiprocessor performance.**
- 2. Enable efficient parallel processing.**
- 3. Prevent hardware and memory conflicts.**
- 4. Increase system reliability.**
- 5. Ensure proper resource utilization.**

Applications

- 1. Multi-core Processors.**
- 2. Supercomputers.**
- 3. Cloud Computing.**
- 4. Database Servers.**
- 5. Artificial Intelligence.**
- 6. Scientific Computing.**
- 7. Distributed Systems.**

Memory in Multiprocessor System

Definition

Memory in a Multiprocessor System refers to the organization and management of memory that is shared or distributed among multiple processors. It allows processors to access data and instructions efficiently while maintaining consistency and synchronization.

The design of memory organization is very important because multiple processors may need to access the same memory simultaneously. Proper memory management improves performance, reduces access time, and prevents data conflicts.

Objectives of Memory Organization in Multiprocessor Systems

- 1. To provide efficient memory access to all processors.**
- 2. To allow processors to share data and resources.**
- 3. To maintain data consistency.**
- 4. To reduce memory access time.**
- 5. To improve overall system performance.**

Types of Memory Organization in Multiprocessor Systems

1. Shared Memory Architecture

Definition

In a Shared Memory Architecture, all processors access a common main memory. Every processor can read and write data from the same memory space, making communication between processors simple and efficient.

Features

- 1. Common memory shared by all processors.**
- 2. Easy communication between processors.**
- 3. Suitable for small and medium multiprocessor systems.**
- 4. Requires synchronization to avoid data conflicts.**

Advantages

- 1. Easy data sharing.**
- 2. Simple programming model.**
- 3. Fast communication between processors.**
- 4. Efficient resource utilization.**

Disadvantages

- 1. Memory contention may occur.**
- 2. Scalability is limited.**
- 3. Bus traffic increases with more processors.**

2. Distributed Memory Architecture

Definition

In a Distributed Memory Architecture, each processor has its own private memory. Processors communicate with each other by sending messages through an interconnection network.

Features

- 1. Separate memory for each processor.**
- 2. Communication through message passing.**
- 3. Suitable for large multiprocessor systems.**
- 4. Better scalability.**

Advantages

- 1. High scalability.**

2. No memory contention.
3. Better performance for large systems.
4. Fault isolation is easier.

Disadvantages

1. Complex programming.
2. Communication overhead.
3. Message passing increases execution time.

Cache Coherence

Definition

Cache Coherence is the mechanism that ensures all processors have the same and updated copy of shared data in their cache memories. If one processor modifies shared data, the updated value must be reflected in the caches of other processors.

Need for Cache Coherence

1. Prevents inconsistent data.
2. Maintains correct program execution.
3. Ensures data reliability.
4. Supports parallel processing.

Memory Consistency

Definition

Memory Consistency refers to the rules that determine the order in which memory operations (read and write) performed by different processors become visible to one another.

It ensures that all processors observe memory updates in a predictable and correct manner.

Advantages of Memory Organization in Multiprocessor Systems

1. Supports parallel processing.
2. Improves system performance.
3. Enables resource sharing.
4. Faster execution of programs.
5. Better utilization of processors.

Disadvantages

1. Complex hardware design.
2. Synchronization overhead.

3. Memory conflicts may occur.
4. Cache coherence management is difficult.
5. Higher implementation cost.

Applications

1. Supercomputers.
2. Cloud Computing.
3. Database Servers.
4. Scientific Research.
5. Artificial Intelligence.
6. Machine Learning.
7. Weather Forecasting.
8. High-Performance Computing (HPC).

Difference Between Shared Memory and Distributed Memory

Shared Memory

All processors share one common memory. Each processor has its own private memory.

Communication through shared memory.

Easy programming.

Limited scalability.

Memory contention may occur.

Suitable for small and medium systems.

Distributed Memory

Communication through message passing.

More complex programming.

Highly scalable.

No shared memory contention.

Suitable for large systems.

Concept of Pipelining

Definition

Pipelining is a technique used in computer architecture to increase the execution speed of the CPU by dividing the execution of an instruction into several stages. Instead of completing one instruction before starting the next, pipelining allows multiple instructions to be processed simultaneously, with each instruction at a different stage of execution.

It is similar to an assembly line in a factory, where different workers perform different tasks on multiple products at the same time.

Basic Concept

In a non-pipelined processor, one instruction is completely executed before the next instruction begins.

In a pipelined processor, several instructions are processed simultaneously. While one instruction is being executed, another instruction is being decoded, and another is being fetched from memory.

This overlapping of instruction execution improves CPU performance and increases instruction throughput.

Stages of Instruction Pipeline

1. Instruction Fetch (IF)

The CPU fetches the instruction from the main memory.

2. Instruction Decode (ID)

The fetched instruction is decoded, and the required operands are identified.

3. Execute (EX)

The Arithmetic Logic Unit (ALU) performs the required arithmetic or logical operation.

4. Memory Access (MEM)

If the instruction requires memory access, data is read from or written to memory.

5. Write Back (WB)

The final result is written back into the destination register.

Working of Pipelining

1. The first instruction is fetched from memory.
2. During the next clock cycle, the first instruction is decoded while the second instruction is fetched.
3. In the third clock cycle, the first instruction is executed, the second is decoded, and the third is fetched.
4. This process continues until all instructions are completed.

Thus, several instructions are executed simultaneously at different stages.

Example

Consider four instructions:

- I_1
- I_2
- I_3
- I_4

Pipeline Execution Table

Clock Cycle Stage 1 (IF) Stage 2 (ID) Stage 3 (EX) Stage 4 (MEM) Stage 5 (WB)

1	I ₁	-	-	-	-
2	I ₂	I ₁	-	-	-
3	I ₃	I ₂	I ₁	-	-
4	I ₄	I ₃	I ₂	I ₁	-
5	-	I ₄	I ₃	I ₂	I ₁
6	-	-	I ₄	I ₃	I ₂
7	-	-	-	I ₄	I ₃
8	-	-	-	-	I ₄

This table shows that multiple instructions are executed simultaneously in different pipeline stages.

Advantages

1. Increases CPU performance.
2. Improves instruction throughput.
3. Reduces overall execution time.
4. Better utilization of CPU resources.
5. Allows multiple instructions to execute simultaneously.
6. Increases system efficiency.

Disadvantages

1. Pipeline hazards may occur.
2. Complex hardware design.
3. Branch instructions reduce performance.
4. Pipeline stalls may occur.
5. Higher implementation cost.

Applications

1. Modern Microprocessors.
2. RISC Processors.
3. Superscalar Processors.
4. Digital Signal Processors (DSP).
5. Graphics Processing Units (GPU).

6. High-Performance Computing Systems.

Difference Between Non-Pipelining and Pipelining

Non-Pipelining

One instruction executes at a time. Multiple instructions execute simultaneously.

Lower CPU utilization.

Slower execution.

Lower throughput.

Simple hardware design.

Pipelining

Higher CPU utilization.

Faster execution.

Higher throughput.

More complex hardware design.

Vector Processing

Definition

Vector Processing is a processing technique in which a single instruction is used to perform the same operation on multiple data elements simultaneously. Instead of processing one data item at a time (scalar processing), vector processing handles an entire vector (array of data) in one operation.

It is mainly used in applications involving large mathematical calculations, such as scientific computing, graphics, weather forecasting, and artificial intelligence.

Features of Vector Processing

1. Processes multiple data elements simultaneously.
2. Uses vector instructions.
3. High-speed computation.
4. Reduces execution time.
5. Efficient for repetitive arithmetic operations.
6. Supports parallel processing.

Working of Vector Processing

1. Data is stored in vector registers.
2. A single vector instruction is issued.
3. The processor performs the same operation on all elements simultaneously.
4. The result is stored back into the vector register.

Example

Suppose,

Vector A = (2, 4, 6, 8)

Vector B = (1, 3, 5, 7)

Vector Addition:

Vector A Vector B Result

2 1 3

4 3 7

6 5 11

8 7 15

Result Vector = (3, 7, 11, 15)

Advantages

- 1. High processing speed.**
- 2. Executes multiple operations simultaneously.**
- 3. Reduces program execution time.**
- 4. Efficient for scientific applications.**
- 5. Better CPU utilization.**

Disadvantages

- 1. Expensive hardware.**
- 2. Complex architecture.**
- 3. Not suitable for irregular data.**
- 4. Requires vectorized programs.**

Applications

- 1. Scientific Computing.**
- 2. Weather Forecasting.**
- 3. Artificial Intelligence.**
- 4. Machine Learning.**
- 5. Image Processing.**
- 6. Computer Graphics.**
- 7. Signal Processing.**
- 8. Supercomputers.**

Array Processing

Definition

Array Processing is a processing technique in which multiple **Processing Elements (PEs)** work together to execute the same instruction on different data elements simultaneously. Each processor performs the same operation on its own data, making array processing a form of parallel processing.

Array processors are designed to improve the performance of applications that involve large arrays or matrices.

Features of Array Processing

1. Consists of multiple processing elements.
2. Supports parallel execution.
3. Executes the same instruction on different data.
4. High computational speed.
5. Suitable for matrix and array operations.

Working of Array Processing

1. Data is divided among multiple processing elements.
2. Each processing element performs the same operation simultaneously.
3. The results from all processors are combined.
4. The final output is produced.

Example

Suppose we want to add two arrays:

Array A = (5, 10, 15, 20)

Array B = (2, 4, 6, 8)

Each processing element performs one addition simultaneously.

Processing Element	Operation	Result
PE ₁	5 + 2	7
PE ₂	10 + 4	14
PE ₃	15 + 6	21
PE ₄	20 + 8	28

Final Result = (7, 14, 21, 28)

Advantages

1. Very high processing speed.
2. Supports large-scale parallel computation.

3. Efficient for matrix operations.
4. Reduces execution time.
5. High throughput.

Disadvantages

1. Complex hardware design.
2. High implementation cost.
3. Difficult programming.
4. Not suitable for sequential tasks.

Applications

1. Matrix Multiplication.
2. Scientific Simulations.
3. Weather Forecasting.
4. Image Processing.
5. Artificial Intelligence.
6. Computer Graphics.
7. Space Research.
8. High-Performance Computing (HPC).

Difference Between Vector Processing and Array Processing

Vector Processing

Uses vector instructions.

One instruction operates on a vector of data.

Uses vector registers.

Suitable for vector calculations.

Hardware is comparatively simpler.

Used in vector processors.

Array Processing

Uses multiple processing elements.

Multiple processors execute the same instruction simultaneously.

Uses an array of processors.

Suitable for matrix and array operations.

Hardware is more complex.

Used in SIMD and array processors.

RISC and CISC

RISC (Reduced Instruction Set Computer)

Definition

RISC (Reduced Instruction Set Computer) is a CPU architecture that uses a small, simple, and highly optimized set of instructions. Each instruction is designed to execute in one clock cycle, making the processor faster and more efficient.

The main objective of RISC is to simplify hardware design and improve execution speed.

Features of RISC

- 1. Uses a small instruction set.**
- 2. Most instructions execute in one clock cycle.**
- 3. Fixed-length instructions.**
- 4. Large number of CPU registers.**
- 5. Load/Store architecture (only load and store instructions access memory).**
- 6. Simple hardware design.**
- 7. Supports efficient pipelining.**

Advantages

- 1. Faster execution speed.**
- 2. Simple processor design.**
- 3. Efficient pipelining.**
- 4. Lower power consumption.**
- 5. Better overall performance.**

Disadvantages

- 1. Larger program size.**
- 2. Requires more memory.**
- 3. Complex software optimization.**

Applications

- 1. Smartphones.**
- 2. Tablets.**
- 3. Embedded Systems.**
- 4. ARM Processors.**
- 5. Modern Mobile Devices.**

Examples

- ARM Processor**
- MIPS Processor**

- **SPARC Processor**
- **RISC-V Processor**

CISC (Complex Instruction Set Computer)

Definition

CISC (Complex Instruction Set Computer) is a CPU architecture that uses a large and complex set of instructions. A single instruction can perform multiple operations, reducing the number of instructions required to execute a program.

The main objective of CISC is to reduce program size by providing powerful instructions.

Features of CISC

- 1. Large instruction set.**
- 2. Variable-length instructions.**
- 3. One instruction may perform multiple tasks.**
- 4. Fewer CPU registers.**
- 5. Memory-to-memory operations are supported.**
- 6. More complex hardware.**
- 7. Higher code density.**

Advantages

- 1. Smaller program size.**
- 2. Fewer instructions required.**
- 3. Easier programming.**
- 4. Better backward compatibility.**

Disadvantages

- 1. Slower execution speed.**
- 2. Complex hardware design.**
- 3. Higher power consumption.**
- 4. Less efficient pipelining.**

Applications

- 1. Personal Computers.**
- 2. Desktop Computers.**
- 3. Laptops.**
- 4. Workstations.**

5. Servers.

Examples

- Intel x86 Processor
- Intel Pentium
- Intel Core Series
- AMD Ryzen Processor

Difference Between RISC and CISC

RISC	CISC
Reduced Instruction Set Computer	Complex Instruction Set Computer
Small instruction set	Large instruction set
Simple instructions	Complex instructions
Fixed instruction length	Variable instruction length
Most instructions execute in one clock cycle	One instruction may require multiple clock cycles
Large number of registers	Fewer registers
Load/Store architecture	Memory-to-memory operations supported
Simple hardware design	Complex hardware design
Efficient pipelining	Less efficient pipelining
Larger program size	Smaller program size
Lower power consumption	Higher power consumption
Faster execution	Comparatively slower execution

Advantages of RISC over CISC

1. Faster instruction execution.
2. Better pipelining.
3. Simpler hardware.
4. Lower power consumption.
5. Higher processing efficiency.

Advantages of CISC over RISC

1. Smaller program size.
2. Fewer instructions required.

3. Easier programming.
4. Better compatibility with older software.

Applications Comparison

RISC	CISC
Mobile Phones	Desktop Computers
Tablets	Laptops
Embedded Systems	Servers
IoT Devices	Workstations

ARM-based Systems Intel and AMD Processors

Study of Multicore Processor – Intel and AMD

Multicore Processor

Definition

A Multicore Processor is a single integrated circuit (IC) that contains two or more independent processing cores. Each core can execute instructions independently, allowing multiple tasks or programs to run simultaneously. Multicore processors improve system performance, increase processing speed, and reduce power consumption compared to using multiple separate processors.

Modern computers, laptops, servers, and smartphones commonly use multicore processors.

Features of Multicore Processor

1. Contains two or more processing cores.
2. Supports parallel processing.
3. Executes multiple tasks simultaneously.
4. Improves multitasking performance.
5. Reduces power consumption.
6. Shares cache memory and system resources.
7. Increases overall system efficiency.

Types of Multicore Processors

1. Dual-Core Processor

Contains 2 processing cores.

2. Quad-Core Processor

Contains 4 processing cores.

3. Hexa-Core Processor

Contains 6 processing cores.

4. Octa-Core Processor

Contains 8 processing cores.

5. Deca-Core and Higher

Contains 10 or more processing cores, mainly used in servers and high-performance systems.

Advantages of Multicore Processors

- 1. Faster execution of programs.**
- 2. Excellent multitasking performance.**
- 3. Supports parallel processing.**
- 4. Lower power consumption than multiple separate processors.**
- 5. Better gaming and multimedia performance.**
- 6. Improved overall system performance.**
- 7. Higher reliability and efficiency.**

Disadvantages of Multicore Processors

- 1. Higher manufacturing cost.**
- 2. Complex processor design.**
- 3. Requires software optimized for multiple cores.**
- 4. Heat generation increases with more cores.**

Intel Multicore Processor

Definition

Intel Multicore Processors are processors developed by Intel Corporation that integrate multiple CPU cores on a single chip. Intel processors are widely used in desktop computers, laptops, workstations, servers, and data centers.

Intel processors are known for high single-core performance, advanced cache technology, Hyper-Threading (on supported models), and Turbo Boost Technology.

Features of Intel Processors

- 1. Multiple processing cores.**
- 2. High processing speed.**
- 3. Large cache memory.**
- 4. Hyper-Threading Technology (supported models).**

5. Turbo Boost Technology.
6. Low power consumption.
7. High reliability.

Advantages

1. Excellent single-core performance.
2. Strong gaming performance.
3. Good software compatibility.
4. Reliable performance.
5. Efficient power management.

Applications

1. Desktop Computers.
2. Laptops.
3. Gaming PCs.
4. Servers.
5. Engineering Workstations.
6. Cloud Computing.

Examples

- Intel Core i3
- Intel Core i5
- Intel Core i7
- Intel Core i9
- Intel Xeon

AMD Multicore Processor

Definition

AMD (Advanced Micro Devices) Multicore Processors are processors designed by AMD that contain multiple CPU cores on a single chip. AMD processors are popular for providing high multi-core performance, competitive pricing, and excellent value for gaming, content creation, and professional workloads.

Modern AMD processors are based on the Zen architecture, offering high efficiency and strong multitasking capabilities.

Features of AMD Processors

1. Multiple processing cores.

2. High multi-core performance.
3. Large cache memory.
4. Simultaneous Multithreading (SMT).
5. Energy-efficient architecture.
6. Excellent performance for parallel applications.

Advantages

1. Excellent multi-core performance.
2. Cost-effective.
3. Good for multitasking.
4. Strong performance in content creation and rendering.
5. High core count in many processor models.

Applications

1. Gaming Computers.
2. Desktop PCs.
3. Laptops.
4. Workstations.
5. Servers.
6. Artificial Intelligence.
7. Scientific Computing.

Examples

- AMD Ryzen 3
- AMD Ryzen 5
- AMD Ryzen 7
- AMD Ryzen 9
- AMD EPYC

Difference Between Intel and AMD Multicore Processors

Intel

Developed by Intel Corporation.

Known for excellent single-core performance.

AMD

Developed by Advanced Micro Devices (AMD).

Known for excellent multi-core performance.

Intel	AMD
Hyper-Threading technology (supported models).	Simultaneous Multithreading (SMT).
Commonly used in business and enterprise systems.	Popular for gaming and content creation.
Intel Xeon processors are used in servers.	AMD EPYC processors are used in servers.
Generally higher price in similar categories.	Often offers better price-to-performance ratio.

Intel vs AMD (Quick Comparison)

Feature	Intel	AMD
Company	Intel Corporation	Advanced Micro Devices (AMD)
Popular Series	Core i3, i5, i7, i9, Xeon	Ryzen 3, 5, 7, 9, EPYC
Performance	Better single-core performance	Better multi-core performance
Multitasking	Very Good	Excellent
Gaming	Excellent	Excellent
Content Creation	Very Good	Excellent
Price	Generally Higher	More Affordable
Server Processor	Xeon	EPYC

Applications of Multicore Processors

1. Gaming.
2. Artificial Intelligence.
3. Machine Learning.
4. Scientific Research.
5. Cloud Computing.
6. Video Editing.
7. 3D Animation.
8. Database Management.
9. Software Development.
- 10.High-Performance Computing (HPC).